SIAM J. SCI. COMPUT.
Vol. 35, No. 2, pp. A740–A766
© 2013 Society for Industrial and Applied Mathematics

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

A FRAMEWORK FOR THE MR^3 ALGORITHM: THEORY AND IMPLEMENTATION*

PAUL R. WILLEMS† AND BRUNO LANG‡

Abstract. This paper provides a streamlined and modular presentation of the MR^3 algorithm for computing selected eigenpairs of symmetric tridiagonal matrices, thus disentangling the principles driving MR^3 and the (recursive) "core" algorithm from the specific (e.g., twisted) decompositions used to represent the matrices at different recursion depths and from the (dqds) transformations converting between them. Our approach allows a modular full proof for the correctness of the MR^3 algorithm. This proof is based on five requirements concerning the interplay between the core algorithm and its subcomponents. These requirements can also guide in implementing the algorithm, because they expose quantities that can and should be monitored at runtime. Our new implementation `XMR`, which is based on the above analysis, is described and compared to `xSTEMR` from LAPACK 3.2.2. Numerical experiments comparing the robustness and performance of both implementations are given.

Key words. symmetric tridiagonal matrix, eigensystem, MRRR algorithm, theory and implementation

AMS subject classifications. 65F15, 65G50, 15A18

DOI. 10.1137/110834020

1. Introduction.

The established strategy for solving a dense real symmetric or complex Hermitian eigenproblem numerically is to reduce the matrix to real tridiagonal form by an orthogonal similarity transformation [1, 16] and then solve the tridiagonal problem. Before there was MR^3, usually one of the following three standard algorithms was employed for the second part.

QR iteration has been the workhorse since the 1960's [14, 15]. The current implementation `xSTEQR` in LAPACK [1] is rock solid. At the time of this writing, the best method for computing all eigenvalues is the dqds algorithm [13, 31], which can be considered to be a specially tailored QR iteration. But for computing eigenvectors, QR exhibits a true $\mathcal{O}(n^3)$ complexity, making it very slow in practice. In fact, for the dense symmetric problem, the solution of the tridiagonal problem using QR can outweigh the reduction phase by far [9]. On the other hand, QR is the only method that allows accumulating the eigenvectors of the original dense problem in place, thus saving a final matrix-matrix multiplication. Together with recently proposed high-performance realizations of sequences of rotations [21, 33], this can make QR (almost) competitive with the approaches discussed below if most or all eigenvectors are required.

Divide and conquer (DC) has been known quite a long time [3], but it took nearly 15 years until a stable implementation was found [18]. Complexity is $\mathcal{O}(n^3)$ in theory, but in practice DC behaves more like $\mathcal{O}(n^{2.5})$ on average [4, 6], due to effective use of BLAS3-operations and a large amount of work that can be saved via deflation.

*Submitted to the journal's Methods and Algorithms for Scientific Computing section May 13, 2011; accepted for publication (in revised form) November 12, 2012; published electronically March 19, 2013. This research was partially funded by the Bundesministerium für Bildung und Forschung, contract 01 IH 08 007 B, within the project *ELPA—Eigenwert-Löser für Petaflop-Anwendungen*.
http://www.siam.org/journals/sisc/35-2/83402.html
†willems@math.uni-wuppertal.de.
‡Faculty of Mathematics and Natural Sciences, University of Wuppertal, D-42097 Wuppertal, Germany (lang@math.uni-wuppertal.de).

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

The current implementation in LAPACK (`xSTEDC`) is fast, accurate, and the method of choice for computing all eigenvalues and eigenvectors. The only major drawback is that $\mathcal{O}(n^2)$ temporary workspace is needed.

In contrast to the preceding two algorithms, *bisection and inverse iteration (BI)* is an *adaptable* method; i.e., it can also compute a subset of k eigenpairs at reduced cost. But due to accuracy limitations of the inverse iteration scheme employed, explicit Gram–Schmidt reorthogonalization of the eigenvectors is required for eigenvalues that are too close to each other (in an absolute sense), so the complexity is $\mathcal{O}(k^2 n)$.

The latest addition to the field has been the algorithm of *multiple relatively robust representations* [7, 8, 9], in short MRRR or MR^3. It offers to compute k eigenpairs $(\bar{\lambda}_i, \bar{\mathsf{q}}_i)$, $\|\bar{\mathsf{q}}_i\| = 1$, of a symmetric tridiagonal matrix $\mathsf{T} \in \mathbb{R}^{n\times n}$ in (optimal) time $\mathcal{O}(kn)$, and thus it is an order of magnitude faster than BI. The adaptability of MR^3, combined with the fact that it requires *no* communication for Gram–Schmidt reorthogonalization, also allows for a large-grained parallelization. The residual norms and orthogonality levels achievable with MR^3 are

$$\|(\mathsf{T} - \bar{\lambda}_i)\bar{\mathsf{q}}_i\| \;=\; \mathcal{O}(\|\mathsf{T}\| n\epsilon_\diamond), \quad |\bar{\mathsf{q}}_i^* \bar{\mathsf{q}}_j| \;=\; \mathcal{O}(n\epsilon_\diamond),\ i \neq j, \tag{1.1}$$

where $\epsilon_\diamond$ denotes machine epsilon; $\epsilon_\diamond = 2^{-53} \approx 1.1 \cdot 10^{-16}$ for IEEE double precision. We use * to denote transposition, $\|\cdot\|$ is the Euclidean norm for vectors and the spectral norm for matrices, and we write $\mathsf{T} - \lambda$ as a shorthand for $\mathsf{T} - \lambda\mathsf{I}$, since we will have to shift matrices all the time.

In a recent study [6] comparing the performance of the current tridiagonal eigensolvers in LAPACK, Demmel et al. made the following observations:

(a) MR^3 does the least amount of floating-point operations across the board.
(b) For not-too-large matrices, DC can be faster than MR^3 if all or most of the eigendecomposition is wanted.
(c) MR^3 is not as accurate as DC or QR in general. The bounds in (1.1) are valid, but for some of their test cases the hidden constant in the $\mathcal{O}$-term lies in the hundreds.

The explanation for the seeming contradiction between (a) and (b) is that MR^3 operates at a lower flop rate. There are two reasons for this. First, a fixed fraction of the flops done by MR^3 are divisions, whereas the proportional amount of divisions in DC goes down with increasing n. And second, DC can be arranged to do most of its work within matrix-matrix multiplications. Their conclusion is that DC may be competitive when the full eigendecomposition is wanted, but for subsets MR^3 is the clear winner.

This paper contributes to the theory of MR^3 by providing a streamlined presentation of the method and to its applicability by describing a new implementation that offers improved robustness while being highly competitive in speed.

Section 2 briefly reviews some concepts that will be needed in the analysis of the MR^3 algorithm. In section 3 we will present the MR^3 algorithm in a new modular way, thus disentangling the principles driving MR^3 and the (recursive) "core" algorithm from the specific decompositions used to represent the matrices at different recursion depths and from the transformations converting between them. As an extension of [9], the modular approach allows us to identify five requirements that are sufficient to guarantee (1.1). The requirements are collected, and a full proof of correctness of the MR^3 algorithm is given in section 4. In section 5 we describe our implementation `XMR` and compare it to `xSTEMR` from LAPACK. Section 6 summarizes the results of our numerical experiments.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

2. Angles and gaps. In this section we briefly recall the concepts of angles between subspaces and of absolute and relative gaps in the spectrum, because these will be needed throughout the analysis of the MR^3 algorithm.

One of the central conditions guaranteeing orthogonality of the computed eigenvectors is based on the angles between subspaces. For $\mathsf{u}, \mathsf{v} \in \mathbb{R}^n$ having unit norm, let $\angle(\mathsf{u}, \mathsf{v}) \in [0, \pi/2]$ denote the acute angle between the one-dimensional subspaces spanned by these vectors, i.e.,

$$\cos\angle(\mathsf{u}, \mathsf{v}) \;=\; |\mathsf{u}^* \mathsf{v}|.$$

This definition is extended to subspaces $\mathcal{U}, \mathcal{V} \subseteq \mathbb{R}^n$ with $0 \le \dim \mathcal{U} \le \dim \mathcal{V}$ via

$$\angle(\mathcal{U}, \mathcal{V}) \;:=\; \max_{\mathsf{u}\in\mathcal{U}} \min_{\mathsf{v}\in\mathcal{V}} \angle(\mathsf{u}, \mathsf{v}).$$

(Interchange the roles of $\mathcal{U}$ and $\mathcal{V}$ if $\dim \mathcal{U} > \dim \mathcal{V}$.) In fact this is just the largest canonical angle, as defined in [16], and $\angle(\mathcal{U}, \mathcal{V}) = 0$ if $\mathcal{U} \subseteq \mathcal{V}$, as it should be.

The following lemma follows immediately from this definition of the angle.

LEMMA 2.1. *Let $\mathcal{U}$, $\mathcal{V}$ be nontrivial subspaces of $\mathbb{R}^n$ with $\mathcal{U} \subseteq \mathcal{V}$. Then for any nontrivial $\mathcal{Z}$*

$$\begin{cases} \angle(\mathcal{U}, \mathcal{Z}) \le \angle(\mathcal{V}, \mathcal{Z}) & \textit{if } \dim \mathcal{V} \le \dim \mathcal{Z}, \\ \angle(\mathcal{Z}, \mathcal{V}) \le \angle(\mathcal{Z}, \mathcal{U}) & \textit{if } \dim \mathcal{Z} \le \dim \mathcal{U}. \end{cases}$$

In section 4.4 we will make use of the following triangle inequality; for a proof see [37, Lemma 1.18].

LEMMA 2.2. *Let $\mathcal{U}$, $\mathcal{V}, \mathcal{W}$ be nontrivial subspaces of $\mathbb{R}^n$ with*

$$\min\{\dim \mathcal{U}, \dim \mathcal{W}\} \;\le\; \dim \mathcal{V} \;\le\; \max\{\dim \mathcal{U}, \dim \mathcal{W}\}.$$

Then

$$\sin\angle(\mathcal{U}, \mathcal{W}) \;\le\; \sin\angle(\mathcal{U}, \mathcal{V}) + \sin\angle(\mathcal{V}, \mathcal{W}).$$

Of particular interest are subspaces spanned by eigenvectors to selected eigenvalues of the matrix. For a symmetric matrix A, let $\lambda_i = \lambda_i[\mathsf{A}]$ denote its ith smallest eigenvalue and $\mathsf{q}_i = \mathsf{q}_i[\mathsf{A}]$ be the corresponding member of an orthonormal basis of eigenvectors, i.e.,

$$\big(\mathsf{A} - \lambda_i[\mathsf{A}]\big)\, \mathsf{q}_i[\mathsf{A}] \;=\; 0, \quad \big\|\mathsf{q}_i[\mathsf{A}]\big\| \;=\; 1.$$

The spectrum of A, $\operatorname{spec} \mathsf{A} = \{\lambda_1[\mathsf{A}], \ldots, \lambda_n[\mathsf{A}]\}$, is the set of all eigenvalues, and for any index set $I \subseteq \{1, \ldots, n\}$, we denote by

$$\mathcal{Q}_I[\mathsf{A}] \;:=\; \operatorname{span}\big\{\mathsf{q}_i[\mathsf{A}] \,\big|\, i \in I\big\}$$

the invariant subspace belonging to eigenvalues with index in I.

Reasoning about the sensitivity of eigenvalues and eigenvectors of symmetric matrices has to take into account the structure of the spectrum, which is manifested by distances between eigenvalues. The total width of the spectrum is called the spectral diameter,

$$\operatorname{spdiam}[A] \;:=\; \lambda_n - \lambda_1.$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

We will now define three notions of *gap* that will be employed repeatedly in the remainder of this paper. Each can be measured in an absolute or relative sense.

For an index set I (not necessarily a set of consecutive numbers), we measure how far eigenvalues associated with that set are from the other ones by

$$\begin{aligned} \mathrm{gap}_{\mathsf{A}}(I) &:= \min\big\{|\lambda_j - \lambda_i| \,:\, i \in I, j \notin I\big\}, \\ \mathrm{relgap}_{\mathsf{A}}(I) &:= \min\big\{|\lambda_j - \lambda_i|/|\lambda_i| \,:\, i \in I, j \notin I\big\}. \end{aligned}$$

Both are considered undefined if $I = \{1, \ldots, n\}$. For the relative gap, we stipulate that $0/0 = 0$ (two zero eigenvalues, one in I and one not) and $x/0 = \infty$ for $x \neq 0$ (simple zero eigenvalue in I).

For the case that we have a sample value μ and want to know the smallest positive distance to an eigenvalue, we will use

$$\begin{aligned} \mathrm{gap}_{\mathsf{A}}(\mu) &:= \min\big\{|\mu - \lambda| \,:\, \mu \neq \lambda \in \mathrm{spec}(\mathsf{A})\big\}, \\ \mathrm{relgap}_{\mathsf{A}}(\mu) &:= \min\big\{|\mu - \lambda|/|\mu| \,:\, \mu \neq \lambda \in \mathrm{spec}(\mathsf{A})\big\}. \end{aligned}$$

Note that the two forms coincide if $\mu = \lambda_i$ is a simple eigenvalue, since then $\mathrm{gap}_{\mathsf{A}}(\mu) = \mathrm{gap}_{\mathsf{A}}(\{i\})$ and $\mathrm{relgap}_{\mathsf{A}}(\mu) = \mathrm{relgap}_{\mathsf{A}}(\{i\})$.

The third form in a sense combines the above two by measuring the distance of a sample value to an eigenvalue not contained in a specified subset,

$$\mathrm{gap}_{\mathsf{A}}(I; \mu) := \min\big\{|\lambda_j - \mu| \,:\, j \notin I\big\}$$

(we will not need a relative version).

3. The core MR^3 algorithm.

From now on our focus is on tridiagonal matrices. For a given symmetric tridiagonal matrix T, we usually name the diagonal entries c_i and the offdiagonals e_i, that is,

$$\mathsf{T} = \mathrm{diag}(c_1, \ldots, c_n) + \mathrm{diag}_{\pm 1}(e_1, \ldots, e_{n-1}) \in \mathbb{R}^{n \times n}, \tag{3.1}$$

where $\mathrm{diag}_k(\cdots)$ means putting the entries on the kth superdiagonal ($k > 0$) or the $(-k)$th subdiagonal ($k < 0$).

There is a long line of research results that led to and contributed to the MR^3 algorithm. The first time, where all pieces were assembled and harnessed fully to form the algorithm we now call MR^3, was in Dhillon's thesis [7]. A completely revised description and a formal proof of correctness followed, distributed across two subsequent publications [8, 9].

The discovery spawned a multitude of further results and developments. We cannot give an exhaustive overview but want to point to some publications concerning relative perturbation theory [26, 27, 28, 30], about issues of relevance for a robust implementation [10, 11, 22], and addressing problematic aspects in the original version and how they can be overcome [2, 34].

In this section we want to develop the driving principles behind MR^3 and then describe the algorithm in full. Our presentation differs in some aspects notably from the original version in the sources mentioned above.

3.1. Preprocessing.

It is advisable to preprocess the given input matrix with regard to a couple of points. First of all, the entries should be scaled properly to fall into practicable range; specific guidelines can be found, for example, in [20]. If some offdiagonal entry is zero, then the matrix is *reducible* to two independent subproblems

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

that can be handled separately. Similarly, if some offdiagonal entries are smaller than $\mathcal{O}(\|\mathsf{T}\|n\epsilon_\diamond)$, then they can safely be set to zero without serious ramifications for the quality of the results in (1.1). The process of eliminating small offdiagonal entries and breaking the problem into unreduced subproblems is called *splitting* the matrix. Once this has been done, one can always find, with $\mathcal{O}(n)$ operations, a diagonal matrix X with entries ± 1 (called a *signature* matrix) such that the offdiagonals of XTX are positive.

Combining these procedures allows us without loss of generality to restrict our attention to matrices T as in (3.1) that obey

$$e_i \;>\; \|\mathsf{T}\|n\epsilon_\diamond, \quad i = 1, \ldots, n-1. \tag{3.2}$$

At this point we want to recall that while all eigenvalues of an unreduced symmetric tridiagonal matrix are simple, they may be arbitrarily close without any of the offdiagonal entries being much smaller than $\|\mathsf{T}\|$; cf. [24, section 7.7].

3.2. The idea. We will restrict ourselves to what we call the *core* algorithm, meaning that we assume reasonable preparations like splitting and scaling have been done, so that we have an unreduced tridiagonal matrix as in (3.2) for which a subset of eigenpairs is to be computed.

From a distant point of view, MR^3 can be seen as a sophisticated variant of inverse iteration. Its salient feature is that no explicit reorthogonalization is needed, resulting in an $\mathcal{O}(n^2)$ complexity. This is achieved, basically, by computing the vectors to such accuracy that they must be orthogonal *automatically*. How this works can be explained easily: If we manage to compute approximate eigenvectors $\bar{\mathsf{q}}_i$ and $\bar{\mathsf{q}}_j$ such that they have small angles to the respective exact eigenvectors,

$$\sin\angle\big(\bar{\mathsf{q}}_i, \mathsf{q}_i\big) \;=\; \mathcal{O}(n\epsilon_\diamond), \qquad \sin\angle\big(\bar{\mathsf{q}}_j, \mathsf{q}_j\big) \;=\; \mathcal{O}(n\epsilon_\diamond), \tag{3.3}$$

then, provided that $\|\bar{\mathsf{q}}_i\| = \|\bar{\mathsf{q}}_j\| = 1$, simple trigonometry gives

$$|\bar{\mathsf{q}}_i^*\bar{\mathsf{q}}_j| \;=\; \cos\angle\big(\bar{\mathsf{q}}_i, \bar{\mathsf{q}}_j\big) \;\le\; \sin\angle\big(\bar{\mathsf{q}}_i, \mathsf{q}_i\big) + \sin\angle\big(\bar{\mathsf{q}}_j, \mathsf{q}_j\big) \;=\; \mathcal{O}(n\epsilon_\diamond). \tag{3.4}$$

The details of this idea will be worked out in the proof of Theorem 4.5 (orthogonality of MR^3) in section 4.4. Right now it will suffice to keep in mind that *accuracy yields orthogonality*.

Accuracy on the levels stipulated in (3.3) is not easy to achieve. Standard inverse iteration can deliver such accuracy only if one is lucky in the choice of the starting vector [19]. In MR^3 there are two principles at work to ascertain success.

Assume the matrix T does "define" an eigenpair (λ, q) that we want to "high relative accuracy"—the precise meaning of these terms will be defined later. Then we are able to compute an approximation $\bar{\lambda}$ of λ with

$$|\bar{\lambda} - \lambda| \;=\; \mathcal{O}(|\lambda| n\epsilon_\diamond) \text{ or even } \mathcal{O}(|\lambda|\epsilon_\diamond).$$

The first ingredient to MR^3 is a new technique that in this situation allows us to compute an approximation $\bar{\mathsf{q}}$ to the eigenvector with a residual norm that is small *compared to the eigenvalue*,

$$\|(\mathsf{T} - \bar{\lambda})\bar{\mathsf{q}}\| \;\le\; \mathcal{O}(|\lambda| n\epsilon_\diamond). \tag{3.5}$$

The key is to exploit the tridiagonal form. Using *twisted factorizations* (discussed toward the end of this subsection) one can determine, in $\mathcal{O}(n)$ time, a position where

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

the true eigenvector has a large entry [12, 29]. Starting with the corresponding canonical vector on the right-hand side, one step of inverse iteration will deliver a residual norm satisfying (3.5), which is at most a factor $\sqrt{n}$ away from being optimal [19]. For details about twisted factorizations and new variants of the dqds transformations relating them in the MR^3 algorithm we refer the reader to [9, 29, 39].

The benefit of achieving (3.5) is revealed by the classical gap theorem.

THEOREM 3.1 (gap theorem). *For every symmetric* $\mathsf{A} \in \mathbb{R}^{n\times n}$*, unit vector* x*, scalar* μ*, and index set* I *such that* $\mathrm{gap}_{\mathsf{A}}(I;\mu) \neq 0$*, we have*

$$\sin\angle\big(\mathsf{x}\,,\,\mathcal{Q}_I[\mathsf{A}]\big) \;\leq\; \frac{\|\mathsf{A}\mathsf{x}-\mathsf{x}\mu\|}{\mathrm{gap}_{\mathsf{A}}(I;\mu)}.$$

For a proof we refer the reader to [24, 37]. Note that (3.5) conveys that $\bar{\lambda}$ must approximate λ to high relative accuracy. Hence, the gap theorem yields

$$\sin\angle\big(\bar{\mathsf{q}},\mathsf{q}\big) \;=\; \mathcal{O}\big(n\epsilon_\diamond/\mathrm{relgap}_{\mathsf{T}}(\lambda)\big). \tag{3.6}$$

Thus, we can reach the goal (3.3) of highly accurate eigenvectors for all eigenvalues whose relative gap to the rest of the spectrum exceeds a constant *gaptol*, the *gap tolerance* (e.g., *gaptol* $\approx 0.001\ldots0.01$ for double precision). Such eigenvalues are called *isolated* or *singletons*; the others are grouped into *clusters*. Care has to be taken when capturing these notions formally, since $\mathrm{relgap}_{\mathsf{T}}(\lambda_i) >$ *gaptol* does not necessarily imply $\mathrm{relgap}_{\mathsf{T}}(\lambda_{i+1}) >$ *gaptol*; see [37] for a detailed discussion.

The second ingredient to MR^3 is the fact that *eigenvectors are shift-invariant, but relative gaps are not.* This allows dealing with eigenvalues that are too close in a relative sense to make the bound (3.6) useful. To exemplify the idea, take a shift $\tau \approx \lambda$ close to an eigenvalue to see that the relative gap with respect to the shifted matrix becomes

$$\mathrm{relgap}_{\mathsf{T}-\tau}(\lambda-\tau) \;=\; \mathrm{relgap}_{\mathsf{T}}(\lambda)\frac{|\lambda|}{|\lambda-\tau|} \gg \mathrm{relgap}_{\mathsf{T}}(\lambda).$$

Thus we can increase relative gaps by shifting close.

Both ingredients taken together give the crude outline for a strategy:

(1) Compute the eigenvalues to reveal relative gaps.
(2) *For all singletons:* Compute the eigenvector such that (3.5) is fulfilled.
(3) *For all clusters:* Shift close to the cluster. This will increase the relative gaps, so at least some should now exceed the tolerance; repeat these three steps.

This looks deceptively simple, but there is a catch. Doing the shifting in floating-point arithmetic will inevitably incur rounding errors so that the eigenvector invariance is lost. To get numerically orthogonal vectors as in (3.4) one must ensure that the computed vectors will still be accurate *with respect to the original matrix.* Hence, we will basically need that rounding errors from the shifting do not affect too severely the invariant subspace we are aiming for.

We can summarize that for the strategy outlined above to work, each encountered shifted matrix has to define the desired eigenvalues and eigenvectors to high relative accuracy, and furthermore the shifting must be done so as not to spoil the invariant subspaces. The key ingredient to overcome both issues is what could be regarded as the third principle underlying MR^3, namely the following: *do not try to compute results to more accuracy than is warranted by the data* (cf. [7, p. 155]). The question of what data one should actually use to define a matrix leads to a crucial new concept.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

DEFINITION 3.2 (representation). *A set of $m \leq 2n-1$ scalars, called the* data, *together with a mapping $f : \mathbb{R}^m \to \mathbb{R}^{2n-1}$ to define the entries of a symmetric tridiagonal matrix $\mathsf{T} \in \mathbb{R}^{n\times n}$ is called a* representation *of* T.

DEFINITION 3.3 (perturbation of a representation). *Let $x_1, \ldots, x_m$ be the scalars used to represent the symmetric tridiagonal matrix T and $\widetilde{x}_i = x_i(1+\xi_i)$ be relative perturbations of them; using the same representational mapping they define a matrix $\widetilde{\mathsf{T}}$.*

If $|\xi_i| \leq \bar{\xi}$ for $i = 1, \ldots, m$, we call $\mathsf{T} \rightsquigarrow \widetilde{\mathsf{T}}$ a componentwise *or* elementwise relative perturbation (erp) *bounded by $\bar{\xi}$, in short $\widetilde{\mathsf{T}} = \mathrm{erp}(\mathsf{T}, \bar{\xi})$.*

A (partial) *relatively robust representation (RRR)* of a matrix T is one where small relative changes in the data, in the form of an erp bounded by some constant $\bar{\xi}$, will cause only relative changes proportional to $\bar{\xi}$ in (some of) the eigenvalues and eigenvectors. We will quantify these notions in section 4.

The obvious way to represent a tridiagonal matrix uses its entries $c_1, \ldots, c_n$, $e_1, \ldots, e_{n-1}$ directly; that is, the representational mapping is just the identity on $\mathbb{R}^{2n-1}$. Unfortunately this will rarely be an RRR, at least not for the small eigenvalues.

Alternatively one can base a representation on a decomposition of the matrix:

- the classical LU-type factorizations $\mathsf{T} = \mathsf{LDL}^*$ with unit lower bidiagonal $\mathsf{L} = \mathrm{diag}(1, \ldots, 1) + \mathrm{diag}_{-1}(\ell_1, \ldots, \ell_{n-1})$ and diagonal $\mathsf{D} = \mathrm{diag}(d_1, \ldots, d_n)$, or $\mathsf{T} = \mathsf{URU}^*$ with unit upper bidiagonal $\mathsf{U} = \mathrm{diag}(1, \ldots, 1) + \mathrm{diag}_{+1}(u_1, \ldots, u_{n-1})$ and diagonal $\mathsf{R} = \mathrm{diag}(r_1, \ldots, r_n)$ (see, e.g., [7]);
- *twisted factorizations* $\mathsf{T} = \mathsf{N}_k \mathsf{G}_k \mathsf{N}_k^*$ that are obtained by combining the upper part of an LDL^* factorization and the lower part of a URU^* factorization:

$$\mathsf{N}_k = \begin{pmatrix} 1 & & & & & & \\ \ell_1 & 1 & & & & & \\ & \ddots & \ddots & & & & \\ & & \ell_{k-1} & 1 & u_{k+1} & & \\ & & & & \ddots & \ddots & \\ & & & & & 1 & u_n \\ & & & & & & 1 \end{pmatrix}, \quad \mathsf{G}_k = \begin{pmatrix} d_1 & & & & & & \\ & \ddots & & & & & \\ & & d_{k-1} & & & & \\ & & & \gamma_k & & & \\ & & & & r_{k+1} & & \\ & & & & & \ddots & \\ & & & & & & r_n \end{pmatrix}$$

  (see, e.g., [7, 29]); or
- block factorizations $\mathsf{T} = \mathsf{LDL}^*$, where D is block diagonal with blocks D_i of size 1×1 or 2×2, and L is block lower bidiagonal with conformant diagonal blocks $\mathsf{L}_i = \begin{pmatrix} 1 \end{pmatrix}$ or $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (see [38]). Analogously to the nonblocked case, one can then also use twisted block factorizations by combining an upper blocked LDL^* part with a lower blocked URU^* part (using a single element as twist).

The seminal paper [5] showed that bidiagonal factorizations of definite matrices will always give RRRs. This result extends to both entrywise representations of tridiagonal matrices with a zero diagonal as well as blocked factorizations of them; matrices with a zero diagonal also provide an example of why it is useful to allow $m < 2n-1$ in the definition above.

In [7] compelling evidence was gathered that LU decompositions $\mathsf{T} = \mathsf{LDL}^*$ will usually result in at least a partial RRR for the smallest eigenvalues; motivated by this fact they were chosen for the original MR^3 algorithm. However, once definiteness is lost, so is the guarantee for robustness. Twisted factorizations perform somewhat better, due to the one more degree of flexibility in choosing the "twist index" k.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Input: Symmetric tridiagonal $\mathsf{T} \in \mathbb{R}^{n\times n}$, index set $I_0 \subseteq \{1,\ldots,n\}$
Output: Eigenpairs $(\bar{\lambda}_i, \bar{\mathsf{q}}_i)$, $i \in I_0$
Params: *gaptol*, the gap tolerance

Find a suitable representation M_0 for T, preferably definite, possibly by shifting T.
$\mathcal{S} := \{(\mathsf{M}_0, I_0, \bar{\tau} = 0)\}$
while $\mathcal{S} \neq \emptyset$ **do**
    Remove one node $(\mathsf{M}, I, \bar{\tau})$ from $\mathcal{S}$.
    Approximate eigenvalues $[\lambda_i^{\mathrm{loc}}]$, $i \in I$, of M such that they can be classified into singletons and clusters according to *gaptol*; this gives a partition $I = I_1 \cup \cdots \cup I_m$.
    for $r = 1$ **to** m **do**
        if $I_r = \{i\}$ **then** // singleton
            Refine eigenvalue approximation $[\lambda_i^{\mathrm{loc}}]$ and use it to compute $\bar{\mathsf{q}}_i$.
            If necessary, iterate until the residual of $\bar{\mathsf{q}}_i$ becomes small enough,
            using a Rayleigh quotient iteration (RQI).
            $\bar{\lambda}_i := \lambda_i^{\mathrm{loc}} + \bar{\tau}$
        else // cluster
            Refine the eigenvalue approximations at the borders of (and/or inside) the cluster if desired for more accurate selection of shift.
            Choose a suitable shift τ near the cluster and compute a representation of $\mathsf{M}^+ = \mathsf{M} - \tau$.
            Add new node $(\mathsf{M}^+, I_r, \bar{\tau} + \tau)$ to $\mathcal{S}$.
        endif
    endfor
endwhile

FIG. 3.1. *Algorithm MR^3: Compute selected eigenvalues and eigenvectors of a symmetric tridiagonal* T.

Block factorizations offer the best option for constructing a robust representation, since arising element growth can be countered dynamically. Their use in the context of MR^3 became possible only recently, when a mixed relatively stable algorithm for shifting them was discovered [37, 38].

On the notational side, we will not explicitly mention the representational mappings from here on but just say T where in fact we mean a representation of T.

3.3. The algorithm and the representation tree. The previously developed ideas are formalized in Algorithm MR^3; see Figure 3.1. One particular nonstandard feature of our version is that nothing is said about the kind of representation used at the nodes. The computed eigenpairs are denoted by $(\bar{\lambda}_i, \bar{\mathsf{q}}_i)$ to distinguish them from the exact ones.

Although we have chosen an iterative layout, the basic nature of the MR^3 algorithm is a recursive one. Therefore, the natural way to model the computation is as traversal of a tree, the *representation tree*. The *nodes* in this tree are the objects placed into and retrieved from the set $\mathcal{S}$; they correspond to the iterations of the outermost loop. Each node has three features associated with it:

- a representation of a tridiagonal matrix M;

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

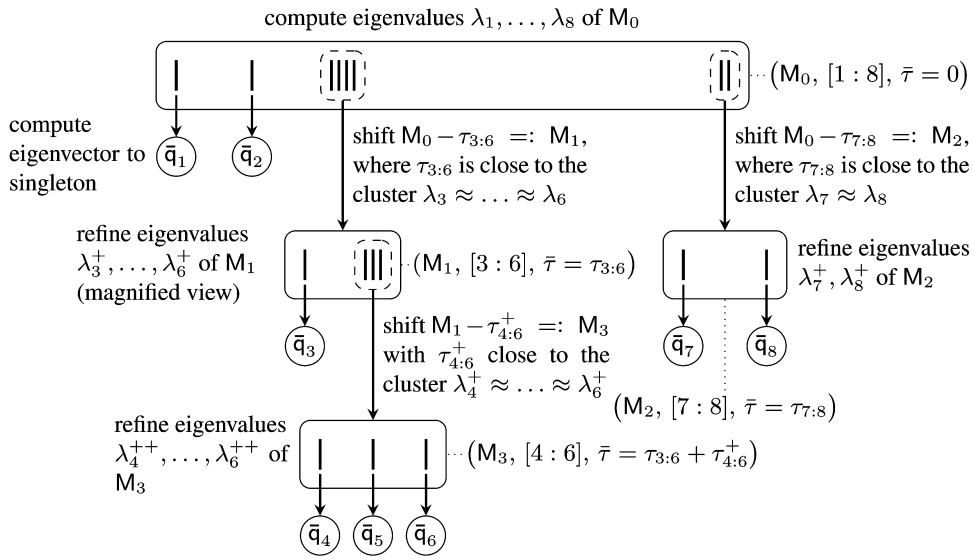

FIG. 3.2. *Example for a representation tree for computing eight eigenpairs. The eigenpair* $(\bar{\lambda}_3, \bar{\mathsf{q}}_3)$ *is computed at node* $(\mathsf{M}_1, [3:6])$.

- an index set I of the eigenpairs that are to be computed for M, called the *local* eigenpairs;
- the accumulated shift $\bar{\tau}$ from the root matrix.

Thus we can write nodes as triplets $(\mathsf{M}, I, \bar{\tau})$; we will sometimes just write (M, I) if the shift is not of interest. The *root* node is $(\mathsf{M}_0, I_0, 0)$ at the top of the tree. See Figure 3.2 for an example of a representation tree.

Line 5 of the algorithm partitions the index set of a node (M, I) as $I = I_1 \cup \cdots \cup I_m$. This basically defines the layout of the tree. Each I_j is called a *child index set* of the node, but only if $|I_j| > 1$ does it lead to a *child node* $(\mathsf{M}_\mathrm{c}, I_j, \bar{\tau}_\mathrm{c})$. Note that this implies that the index sets of all nodes except possibly the root have at least two elements.

For each computed eigenpair there is exactly one singleton child index set of a node somewhere. Conceptually we regard these as the *leaves* of the tree, but they are *not* nodes in the above sense, because no representation is attached. If $\{i\}$ is a child index set of (M, I), we say that the eigenpair $(\bar{\lambda}_i, \bar{\mathsf{q}}_i)$ was *computed at* the node (M, I).

For each node we define $\mathrm{d}(\mathsf{M})$ to be its *depth*, that is, the number of edges on the unique path leading from the root to it, so $\mathrm{d}(\mathsf{M}_0) = 0$. Furthermore, for each $j \in I_0$ let $\mathrm{d}(j)$ stand for the depth of the node where the eigenpair $(\bar{\lambda}_j, \bar{\mathsf{q}}_j)$ is computed. (If we were to regard leaves $\{j\}$ as nodes, their depth would be $\mathrm{d}(j) + 1$.)

Note that it is possible that a child node has the same index set as its parent if a cluster does not split, but a node is uniquely identified by the combination of index set and depth.

4. Proof of correctness. We now set out to present, from scratch, a complete error analysis of the MR^3 algorithm. Originally this was presented by the inventors in [8, 9]; what is to follow will combine and, to some degree, extend their work.

The basic strategy we use is the same as in [8]: Identify a set of (reasonable and

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

practicable) requirements that are assumed to be fulfilled and on whose shoulders the proof can rest. The advantage of this approach is that it provides, as a side effect, a concise set of guidelines for an implementation, e.g., the conditions that have to be heeded when selecting a shift in line 12.

Our presentation is more abstract, because we do not specify the type of representation used. Since the beginning, it has been clear to everyone involved with MR^3 that one could in principle use different kinds of representations; it just had not been justified formally. In fact, in [8, 9] the proof was tightly intertwined with the use of bidiagonal and twisted factorizations. By contrast, our analysis will show that these particular factorizations do not need to be an integral part of the algorithm per se. Indeed, block factorizations might be used instead; see [38]. Furthermore, the line of argumentation has been compressed and streamlined.

Recall that $(\bar{\lambda}_i, \bar{\mathsf{q}}_i)$ are the eigenpairs computed by Algorithm MR^3. We can make the simplifying assumption that the computed vectors are normed to unity *exactly*, $\|\bar{\mathsf{q}}_i\| \equiv 1$. Otherwise just replace $\bar{\mathsf{q}}_i$ by $\bar{\mathsf{q}}_i/\|\bar{\mathsf{q}}_i\|$ everywhere.

4.1. Overview of the proof. The two goals will be to obtain error bounds for the final residuals and orthogonality levels.

The proof of the residual bound in Theorem 4.1 is based on the following reasoning. As an example, consider the eigenvector $\bar{\mathsf{q}}_5$ computed at node $(\mathsf{M}_3, [4:6])$ in Figure 3.2. Assume that the eigenpair can be computed with a "sufficiently small" residual with respect to that node's matrix, represented by M_3 (a precise formulation for this will be given as requirement Getvec in section 4.2), and that the shift from the parent node $(\mathsf{M}_1, [3:6])$ to $(\mathsf{M}_3, [4:6])$ has been performed in a stable way (requirement Shiftrel); then $\bar{\mathsf{q}}_5$ will also provide a small residual for the parent node. Similarly, if the latter (Shiftrel) holds for all transitions on the way down from the root, then a small residual with respect to the original matrix can be guaranteed. The proof of Theorem 4.1 actually requires an additional property, namely that the entries of the intermediate matrices M_j not be overly sensitive to perturbations in the representations (requirement Elg).

The proof of orthogonality in Theorem 4.5 relies on the exact eigenvectors being orthogonal and being close to the computed ones. While the former is a direct consequence of irreducibility, the latter is proved by induction. Lemma 4.2 establishes the induction basis, namely that the computed eigenvectors are close to the exact ones at the respective node, e.g., node $(\mathsf{M}_3, [4:6])$ for $\bar{\mathsf{q}}_5$, provided that we have a small residual (Getvec, again), that the classification into singletons and clusters is done in a reasonable way (requirement Relgaps), and that the eigenvectors are not too sensitive to perturbations of the representations (requirement RRR). The induction step is made in Lemma 4.3, which claims that the invariant subspace associated with an index set does not change much from a child to its parent node if the representation is relatively robust and the classification and the shifting were done properly (RRR, Relgaps, and Shiftrel). Then Lemma 4.4 concludes that the computed vectors are close to the local invariant subspaces all along the way from the root, before Theorem 4.5 wraps everything up.

This gives five distinct requirements a computational run of MR^3 has to fulfill in order to achieve these goals; they are compiled in section 4.2 before we come to the proofs.

4.2. A list of requirements. The first three requirements (RRR, Elg, Relgaps) can be found in similar forms in [8, pp. 10–12]. How to attain the fourth (Shiftrel) and fifth (Getvec) requirements for bidiagonal factorizations is the topic

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

of [9]; stating them as requirements provides the capsule in which we hide the particular kind of representation used.

Requirement 1: RRR (relatively robust representation)
There is a constant C_{vecs} *such that for any perturbation* $\widetilde{\mathsf{M}} = \operatorname{erp}(\mathsf{M}, \alpha)$ *at a node* (M, I)*, the effect on the eigenvectors can be controlled as*

$$\sin\angle\big(\mathcal{Q}_J[\mathsf{M}]\,,\,\mathcal{Q}_J[\widetilde{\mathsf{M}}]\big) \;\le\; C_{\text{vecs}}\, n\alpha \big/ \operatorname{relgap}_{\mathsf{M}}(J)$$

for all $J \in \{I, I_1, \ldots, I_m\}$ *with* $|J| < n$.

(Recall that m and the sets I_r are set in line 5 of Algorithm MR^3.) This requirement was already motivated in section 3.2, namely that the local invariant subspaces have to be relatively robust in the face of small changes to a node's representation data.

The stated condition concerns only eigenvectors, because that is all we need explicitly. We do *not* need that each eigenvalue within a cluster has to be relatively robust. Indeed, such a requirement would be very hard to guarantee for a cluster with moderate relative width (for example, $\approx \sqrt{\epsilon_\diamond}$). In that regard our formulation differs notably from [8]. However, using the gap theorem, Theorem 3.1, we see that fulfillment of the stated condition implies that the *boundaries* of a cluster, as well as the singleton eigenvalues, will be relatively robust, since they cannot change by more than $\mathcal{O}(C_{\text{vecs}} n\alpha|\lambda|)$.

Requirement 2: ELG (conditional element growth)
There is a constant C_{elg} *such that for any perturbation* $\widetilde{\mathsf{M}} = \operatorname{erp}(\mathsf{M}, \alpha)$ *at a node* (M, I)*, the incurred element growth is bounded by*

$$\begin{aligned} \|\widetilde{\mathsf{M}} - \mathsf{M}\| &\le \operatorname{spdiam}[\mathsf{M}_0], \\ \|(\widetilde{\mathsf{M}} - \mathsf{M})\bar{\mathsf{q}}_i\| &\le C_{\text{elg}} n\alpha \operatorname{spdiam}[\mathsf{M}_0] \qquad \textit{for each } i \in I. \end{aligned}$$

This requirement concerns the *absolute* changes to matrix entries that result from *relative changes* to the representation data. For decomposition-based representations this is called *element growth (elg)*. Note that the requirement becomes moot if the matrix is represented by its entries directly.

The formulation of the two conditions that constitute the requirement are quite deliberately chosen. Combined they convey that even huge element growth is permissible (first condition), but only in those entries where the local eigenvectors of interest have tiny entries (second condition). This meaning is not really affected by the fact that the second condition is stated using the *computed* vectors (as done in [8] as well) instead of the exact ones. An advantage of doing it this way is that it provides a direct and easily checkable criterion for seeing whether the computation is on the right track. In fact one could still do the overall proof if $\bar{\mathsf{q}}_i$ were replaced by q_i here, but it would require more work of a purely technical nature.

A final note concerning element growth. In the requirement it is measured globally with respect to the spectral diameter (of the root). A stronger condition would compare $\widetilde{\mathsf{M}}(i,j)$ to $\mathsf{M}(i,j)$ directly (in particular the diagonal entries) to evaluate what we like to call *local* element growth, but that is not necessary at this point. Dhillon conjectured [7, p. 123] that absence of local element growth implies relative

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

robustness, at least for the eigenvalues of small magnitude, but as of now this claim has not been proved. We will come back to the connection between element growth and relative robustness in section 5.

Requirement 3: RELGAPS (relative gaps)
For each node (M, I)*, the classification of* I *into child index sets in line* 5 *is done such that for* $r = 1, \ldots, m$*,* $\mathrm{relgap}_{\mathsf{M}}(I_r) \geq$ *gaptol (if* $|I_r| < n$*).*

This requirement conveys that the structure of the representation tree has to be consistent with our definition of relative gaps. With regard to the coming proof, RELGAPS is the key to allowing the use of both RRR and GETVEC. It influences two separate aspects of Algorithm MR^3.

The first is of course the refinement of eigenvalues in line 5. But it should be clear that if the eigenvalues are approximated accurately enough and the classification is done sensibly, then fulfillment of the requirement should not be an issue. The other concern is slightly hidden: The requirement also touches on the *outer* relative gaps of the whole local subset at the node. The reason is that

$$\max_r \left\{\mathrm{relgap}_{\mathsf{M}}(I_r)\right\} \;\leq\; \mathrm{relgap}_{\mathsf{M}}(I),$$

so fulfillment of the requirement implies $\mathrm{relgap}_{\mathsf{M}}(I) \geq$ *gaptol*; that is, the requirement cannot be fulfilled if $\mathrm{relgap}_{\mathsf{M}}(I) <$ *gaptol*. This fact has to be kept in mind when the node is created, in particular during evaluation of shifts for a new child in line 12. Furthermore, it has to hold for the root node and I_0. If it does not when only a subset of eigenpairs is desired, $I_0 \subsetneq \{1, \ldots, n\}$, one has to deploy special countermeasures to deal with this problem [22].

Requirement 4: SHIFTREL (shift relation)
There exist constants $\alpha_\downarrow, \alpha_\uparrow$ *such that for every node with matrix* H *that was computed using shift* τ *as child of* M*, there are perturbations*

$$\breve{\mathsf{M}} = \mathrm{erp}(\mathsf{M}, \alpha_\downarrow) \quad \textit{and} \quad \widehat{\mathsf{H}} = \mathrm{erp}(\mathsf{H}, \alpha_\uparrow)$$

with which the exact shift relation $\breve{\mathsf{M}} - \tau \;=\; \widehat{\mathsf{H}}$ *is attained.*

This requirement provides the glue that connects the nodes in the tree. It basically states that the computations of the shifted representations have to be done in a mixed relatively stable way. This is fulfilled, for example, when using twisted factorizations combined with qd-transformations, as described in [9]. Improved variants of these techniques and a completely new approach based on block decompositions are presented in [38, 39].

Note that the perturbation $\breve{\mathsf{M}} = \mathrm{erp}(\mathsf{M}, \alpha_\downarrow)$ at the parent will in general be different for each of its child nodes.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Requirement 5: GETVEC (computation of eigenvectors)
There exist constants $\alpha_\ddagger$, $\beta_\ddagger$, and R_{gv} with the following property: Let $(\bar\lambda^{\mathrm{leaf}}, \bar{\mathsf{q}})$ with $\bar{\mathsf{q}} = \bar{\mathsf{q}}_i$ be computed at node (M, I), where $\bar\lambda^{\mathrm{leaf}}$ is the final local eigenvalue approximation. Then we can find elementwise perturbations to the matrix and the vector,

$$\widetilde{\mathsf{M}} = \mathrm{erp}(\mathsf{M}, \alpha_\ddagger), \qquad \tilde{\mathsf{q}}(j) = \bar{\mathsf{q}}(j)(1+\beta_j) \text{ with } |\beta_j| \le \beta_\ddagger,$$

for which the residual norm is bounded as

$$\|\mathsf{r}^{\mathrm{leaf}}\| \;:=\; \|(\widetilde{\mathsf{M}} - \bar\lambda^{\mathrm{leaf}})\tilde{\mathsf{q}}\| / \|\tilde{\mathsf{q}}\| \;\le\; R_{\mathrm{gv}} n \epsilon_\diamond \,\mathrm{gap}_{\widetilde{\mathsf{M}}}(\{i\}; \bar\lambda^{\mathrm{leaf}}).$$

This final requirement captures what was postulated at the beginning of section 3.2, in (3.5), namely that the vectors computed in line 8 have to have residual norms that are small, even when compared to the eigenvalue.

The discovery of the technique making achievement of this requirement possible was a major breakthrough on the road to MR^3, building on pioneering work by Fernando, Parlett, and Dhillon (see [9, 12, 29]). It revolves around computing a twisted factorization

$$\mathsf{M} - \bar\lambda \;=\; \mathsf{N}_k \mathsf{G}_k \mathsf{N}_k^* \tag{4.1}$$

and then solving $\mathsf{N}_k \mathsf{G}_k \mathsf{N}_k^* \bar{\mathsf{q}} = \gamma_k \mathsf{e}_k$ for the eigenvector. The central result is that one can always determine, in $\mathcal{O}(n)$ time, a twist index k that sufficiently exposes the near-singularity of $\mathsf{M} - \bar\lambda$, having $|\gamma_k| = \mathcal{O}(n|\bar\lambda - \lambda|)$. Further crucial ingredients are qd-type transformations to execute the shift in (4.1) with mixed relative stability and the fact that due to the bidiagonal structure of N_k, the solution of $\mathsf{N}_k \mathsf{G}_k \mathsf{N}_k^* \bar{\mathsf{q}} = \gamma_k \mathsf{e}_k$ involves only multiplications. In particular, the latter is the reason why requirement GETVEC has to allow for perturbations in the entries of the computed vector.

It may seem strange that the required residual bound is given in terms of the gap (of the perturbed matrix) instead of the eigenvalue. In the situation at hand we are talking about a singleton eigenvalue, meaning that the gap will not be much smaller in magnitude than the eigenvalue. Assuming $\bar\lambda = \bar\lambda^{\mathrm{leaf}}$ approximates $\lambda = \lambda_i[\mathsf{M}]$ to high relative accuracy, that is, $\bar\lambda = \lambda(1 + \mathcal{O}(n\epsilon_\diamond))$, for all intents and purposes we will have $\mathrm{gap}_{\widetilde{\mathsf{M}}}(\bar\lambda) \sim \mathrm{gap}_{\mathsf{M}}(\lambda)$ and therefore

$$|\bar\lambda|\, \mathit{gaptol} \lesssim \mathrm{gap}_{\widetilde{\mathsf{M}}}(\bar\lambda).$$

Hence we could also have required a residual norm of $\mathcal{O}(n\epsilon_\diamond|\bar\lambda|\mathit{gaptol})$. However, formulating the condition based on the gap eases access to the gap theorem, Theorem 3.1. Furthermore, quite often we have the situation that the eigenvalue of interest has a relative separation far better than *gaptol*. Then this formulation clearly states that high relative accuracy in the eigenvalue is not strictly necessary. As such, it also provides a practicable convergence criterion for the iteration in line 8.

The requirements declare a small set of constants to control the error bounds globally; an overview of them is given in Table 4.1, together with reasonable assumptions about their sizes for an average test case. Bounds for $\alpha_\downarrow$ and $\alpha_\uparrow$ can be obtained from mixed error analyses of the transformations [38, 39]; the others arise from experience.

4.3. Residual norms. In the following we will derive a first-order bound for the residual norms of the computed vectors $\bar{\mathsf{q}}_i$ with respect to the root matrix and

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 4.1

Summary of parameters in the requirements for MR^3. All are expected to be uniform over the whole tree. The rightmost column gives magnitudes of what might come up in practice for an average problem.

Name	Source	Function	Expected Size
C_{vecs}	RRR	controls change in eigenvectors	moderate (≈ 10)
C_{elg}	ELG	factor for element growth	moderate (≈ 10)
$\alpha_\downarrow$	SHIFTREL	erp at parent to link with child	$\mathcal{O}(\epsilon_\diamond)$
$\alpha_\uparrow$	SHIFTREL	erp at child to link with parent	$\mathcal{O}(\epsilon_\diamond)$
$\alpha_\ddagger$	GETVEC	erp for vector computations	$\mathcal{O}(\epsilon_\diamond)$
$\beta_\ddagger$	GETVEC	relative change to vector entries	$\mathcal{O}(n\epsilon_\diamond)$
R_{gv}	GETVEC	factor for the residual bound	up to $\mathcal{O}(1/gaptol)$

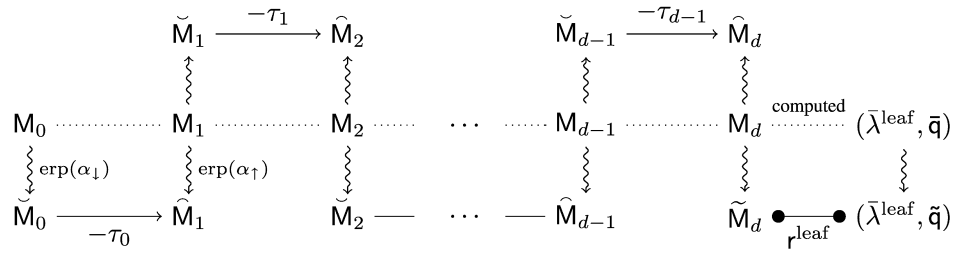

FIG. 4.1. *The computed eigenpairs relate to the root along an alternating sequence of matrix perturbations and exact shift relations.*

an ideal eigenvalue approximation λ^* which one would get if the shift accumulations in lines 9 and 13 were done exactly. For the actually returned eigenvalue $\bar\lambda_i$, the bound would have to be adjusted minimally to take into account the rounding errors from adding the shifts together. But since, for a given vector x, its Rayleigh quotient $\rho_{\mathsf{A}}(\mathsf{x}) := \mathsf{x}^*\mathsf{A}\mathsf{x}/\mathsf{x}^*\mathsf{x}$ is the best approximation to any eigenvalue of A, it might be a better idea to return the Rayleigh quotient of the computed eigenvector instead of the eigenvalue $\bar\lambda_i$ (accumulated according to line 9) in order to minimize the residual norm. The residual r^{leaf} used in the statement of the theorem is the one with respect to the local eigenvalue $\bar\lambda^{\text{leaf}}$, as specified by GETVEC.

THEOREM 4.1 (residual norms for MR^3). *Let the representation tree traversed by Algorithm MR^3 satisfy requirements* ELG, SHIFTREL, *and* GETVEC.

For a given index $j \in I_0$, let $d = \mathrm{d}(j)$ be the depth of the node where $\bar{\mathsf{q}} = \bar{\mathsf{q}}_j$ was computed (cf. section 3.3*) and let $\mathsf{M}_0, \mathsf{M}_1, \ldots, \mathsf{M}_d$ be the representations along the path from the root (M_0, I_0) to that node, with shifts τ_i linking M_i and M_{i+1}, respectively (cf. Figure* 4.1*).*

Define $\lambda^* := \tau_0 + \cdots + \tau_{d-1} + \bar\lambda^{\text{leaf}}$. Then

$$\left\|(\mathsf{M}_0 - \lambda^*)\bar{\mathsf{q}}\right\| \;\le\; \left(\left\|\mathsf{r}^{\text{leaf}}\right\| + \gamma\,\mathrm{spdiam}[\mathsf{M}_0]\right)\frac{1+\beta_\ddagger}{1-\beta_\ddagger},$$

where $\gamma := C_{\text{elg}}\, n\left(d(\alpha_\downarrow + \alpha_\uparrow) + \alpha_\ddagger\right) + 2(d+1)\beta_\ddagger$.

Proof. Requirement SHIFTREL gives us perturbations along the path,

$$\breve{\mathsf{M}}_k = \mathrm{erp}(\mathsf{M}_k, \alpha_\downarrow) \quad \text{and} \quad \widehat{\mathsf{M}}_{k+1} = \mathrm{erp}(\mathsf{M}_{k+1}, \alpha_\uparrow),$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

such that

$$\breve{\mathsf{M}}_k - \tau_k \;=\; \widehat{\mathsf{M}}_{k+1}, \quad k = 0, \ldots, d-1.$$

Furthermore, GETVEC specifies another perturbed matrix

$$\widetilde{\mathsf{M}}_d \;=\; \operatorname{erp}(\mathsf{M}_d, \alpha_\ddagger), \qquad \tilde{\mathsf{q}} \;=\; \bar{\mathsf{q}} + \delta\bar{\mathsf{q}},\ \|\delta\bar{\mathsf{q}}\| \le \beta_\ddagger,$$

with $\mathsf{r}^{\text{leaf}} = (\widetilde{\mathsf{M}}_d - \bar{\lambda}^{\text{leaf}})\tilde{\mathsf{q}}/\|\tilde{\mathsf{q}}\|$. The complete situation is depicted in Figure 4.1. We can link the representations on the path using a telescope sum:

$$\begin{aligned}
\mathsf{M}_0 - \lambda^* &= \mathsf{M}_d - \bar{\lambda}_{\text{leaf}} + \sum_{k=0}^{d-1}\big[(\mathsf{M}_k - \tau_k) - \mathsf{M}_{k+1}\big] \\
&= \mathsf{M}_d - \bar{\lambda}_{\text{leaf}} + \sum_{k=0}^{d-1}\big[(\mathsf{M}_k - \breve{\mathsf{M}}_k) + \underbrace{(\breve{\mathsf{M}}_k - \tau_k) - \widehat{\mathsf{M}}_{k+1}}_{=0} + (\widehat{\mathsf{M}}_{k+1} - \mathsf{M}_{k+1})\big] \\
&= (\widetilde{\mathsf{M}}_d - \bar{\lambda}_{\text{leaf}}) + (\mathsf{M}_d - \widetilde{\mathsf{M}}_d) + \sum_{k=0}^{d-1}\big[(\mathsf{M}_k - \breve{\mathsf{M}}_k) + (\widehat{\mathsf{M}}_{k+1} - \mathsf{M}_{k+1})\big].
\end{aligned}$$

Multiply by $\tilde{\mathsf{q}} = \bar{\mathsf{q}} + \delta\bar{\mathsf{q}}$ and take norms. Then employ $\|\delta\bar{\mathsf{q}}\| \le \beta_\ddagger$, $\|\tilde{\mathsf{q}}\| \le 1 + \beta_\ddagger$, and the bounds provided by GETVEC and ELG to obtain

$$\begin{aligned}
\|(\mathsf{M}_0 - \lambda^*)\tilde{\mathsf{q}}\| \;\le\;& \|\mathsf{r}^{\text{leaf}}\|(1 + \beta_\ddagger) \\
&+ C_{\text{elg}}\, n\,\big(d(\alpha_\downarrow + \alpha_\uparrow) + \alpha_\ddagger\big)\operatorname{spdiam}[\mathsf{M}_0] \\
&+ \beta_\ddagger\big(2d + 1\big)\operatorname{spdiam}[\mathsf{M}_0] \\
=\;& \|\mathsf{r}^{\text{leaf}}\|(1 + \beta_\ddagger) + (\gamma - \beta_\ddagger)\operatorname{spdiam}[\mathsf{M}_0] \\
=:\;& \Delta.
\end{aligned}$$

Now we need only replace $\tilde{\mathsf{q}}$ by $\bar{\mathsf{q}}$ on the left. By a well-known result (cf. Theorem 4.5.1 in [24]) the above bound on the residual norm implies the existence of an eigenvalue λ of M_0 with $|\lambda - \lambda^*| \le \Delta/\|\tilde{\mathsf{q}}\|$. This gives

$$\begin{aligned}
\|(\mathsf{M}_0 - \lambda^*)\bar{\mathsf{q}}\| \;&\le \Delta + \|(\mathsf{M}_0 - \lambda^*)\delta\bar{\mathsf{q}}\| \\
&\le \Delta + \|(\mathsf{M}_0 - \lambda)\delta\bar{\mathsf{q}}\| + \|(\lambda - \lambda^*)\delta\bar{\mathsf{q}}\| \\
&\le \Delta + \beta_\ddagger \operatorname{spdiam}[\mathsf{M}_0] + \Delta\|\delta\bar{\mathsf{q}}\|/\|\tilde{\mathsf{q}}\| \\
&\le \big(\Delta + \beta_\ddagger \operatorname{spdiam}[\mathsf{M}_0]\big)\big(1 + \|\delta\bar{\mathsf{q}}\|/\|\tilde{\mathsf{q}}\|\big),
\end{aligned}$$

and noting that $\|\delta\bar{\mathsf{q}}\|/\|\tilde{\mathsf{q}}\| \le \beta_\ddagger/(1 - \beta_\ddagger)$ the claim follows. ∎

4.4. Orthogonality. For proving that the computed vectors are numerically orthogonal, it will again be necessary to propagate them up the tree. We will make frequent use of the concept of angles between subspaces, in particular the triangle inequality in Lemma 2.2. The key is to show that each $\bar{\mathsf{q}}_i$ has a small angle to *all* invariant subspaces $\mathcal{Q}_I[\mathsf{M}]$ of ancestors in the tree, which are just all nodes (M, I) with $I \ni i$. Note that if the shifting as well as computing of $\bar{\mathsf{q}}_i$ were done in exact arithmetic, we would have containment, $\bar{\mathsf{q}}_i \in \mathcal{Q}_I[\mathsf{M}]$, so all these angles would be zero.

We start by harnessing GETVEC to establish that each computed vector is close to the corresponding exact eigenvector at the node where it is computed.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

LEMMA 4.2 (accuracy of computed vectors). *Let the representation tree traversed by Algorithm MR^3 satisfy requirements* GETVEC, RRR, *and* RELGAPS. *Then a vector* $\bar{\mathsf{q}}_i$ *computed at node* (M, I) *will obey*

$$\sin\angle\big(\bar{\mathsf{q}}_i\,,\,\mathsf{q}_i[\mathsf{M}]\big) \;\le\; C_{\text{vecs}}\alpha_{\ddagger} n/\mathit{gaptol} + R_{\text{gv}} n\epsilon_{\diamond} + \beta_{\ddagger} \;=:\; \mathcal{R} n\epsilon_{\diamond}, \tag{4.2}$$

defining $\mathcal{R}$.

Proof. Let $\widetilde{\mathsf{M}}$ and $\tilde{\mathsf{q}}$ be the perturbed matrix and vector specified by GETVEC. Then we can use Lemma 2.2 for

$$\sin\angle\big(\bar{\mathsf{q}}_i\,,\,\mathsf{q}_i[\mathsf{M}]\big) \;\le\; \sin\angle\big(\bar{\mathsf{q}}_i, \tilde{\mathsf{q}}_i\big) + \sin\angle\big(\tilde{\mathsf{q}}_i\,,\,\mathsf{q}_i[\widetilde{\mathsf{M}}]\big) + \sin\angle\big(\mathsf{q}_i[\widetilde{\mathsf{M}}]\,,\,\mathsf{q}_i[\mathsf{M}]\big).$$

We will proceed by considering each of the three summands on the right separately. Plugging the residual bound from GETVEC into the gap theorem, Theorem 3.1, yields

$$\sin\angle\big(\tilde{\mathsf{q}}_i\,,\,\mathsf{q}_i[\widetilde{\mathsf{M}}]\big) \;\le\; R_{\text{gv}} n\epsilon_{\diamond},$$

which takes care of the middle term. If $\bar{\mathsf{q}}_i$ is computed at node (M, I), then i must be a singleton eigenvalue; i.e., $\{i\}$ is a child index set of I. Thus, by requirement RELGAPS, its relative gap exceeds *gaptol*. Consequently, requirement RRR allows us to bound the third term as

$$\sin\angle\big(\mathsf{q}_i[\widetilde{\mathsf{M}}]\,,\,\mathsf{q}_i[\mathsf{M}]\big) \;\le\; C_{\text{vecs}}\alpha_{\ddagger} n/\mathit{gaptol}.$$

Finally we have to consider the componentwise perturbation $\bar{\mathsf{q}}_i \rightsquigarrow \tilde{\mathsf{q}}_i$ to the computed vector's entries. This we can write as $\tilde{\mathsf{q}}_i = \mathsf{X}\bar{\mathsf{q}}_i$, where X is diagonal holding the individual relative perturbations $1+\beta_j$ from GETVEC. But then the triangle inequality gives

$$\sin\angle\big(\bar{\mathsf{q}}_i, \tilde{\mathsf{q}}_i\big) \;\le\; \|(\mathsf{X}-\mathsf{I})\bar{\mathsf{q}}_i\| \;\le\; \|\mathsf{X}-\mathsf{I}\| \;\le\; \beta_{\ddagger}. \qquad \square$$

The next result provides control over the shifts, as it shows how to relate an invariant subspace of a child to the corresponding subspace at the parent.

LEMMA 4.3 (invariant subspace relations). *Let the representation tree traversed by Algorithm MR^3 satisfy requirements* RRR, SHIFTREL, *and* RELGAPS. *Then for each node* (M, I) *with child* (H, J),

$$\sin\angle\big(\mathcal{Q}_J[\mathsf{M}]\,,\,\mathcal{Q}_J[\mathsf{H}]\big) \;\le\; C_{\text{vecs}}(\alpha_{\downarrow} + \alpha_{\uparrow}) n/\mathit{gaptol}.$$

Proof. Let τ be the shift used to compute H, and let $\breve{\mathsf{M}}$, $\widehat{\mathsf{H}}$ be the perturbed versions of parent and child nodes, as specified by requirement SHIFTREL. Then $\widehat{\mathsf{H}} = \breve{\mathsf{M}} - \tau$, and so $\mathcal{Q}_J[\breve{\mathsf{M}}] = \mathcal{Q}_J[\widehat{\mathsf{H}}]$. Hence, Lemma 2.2 gives

$$\sin\angle\big(\mathcal{Q}_J[\mathsf{M}]\,,\,\mathcal{Q}_J[\mathsf{H}]\big) \;\le\; \sin\angle\big(\mathcal{Q}_J[\mathsf{M}]\,,\,\mathcal{Q}_J[\breve{\mathsf{M}}]\big) + \sin\angle\big(\mathcal{Q}_J[\widehat{\mathsf{H}}]\,,\,\mathcal{Q}_J[\mathsf{H}]\big).$$

Now, as J is a child index set of I, RELGAPS implies $\text{relgap}_{\mathsf{M}}(J) \ge \mathit{gaptol}$ as well as $\text{relgap}_{\mathsf{H}}(J) \ge \mathit{gaptol}$. With the help of requirement RRR we can therefore bound the two terms on the right in the inequality above separately as

$$\begin{aligned}\sin\angle\big(\mathcal{Q}_J[\mathsf{M}]\,,\,\mathcal{Q}_J[\breve{\mathsf{M}}]\big) &\;\le\; C_{\text{vecs}}\alpha_{\downarrow} n/\mathit{gaptol},\\ \sin\angle\big(\mathcal{Q}_J[\mathsf{H}]\,,\,\mathcal{Q}_J[\widehat{\mathsf{H}}]\big) &\;\le\; C_{\text{vecs}}\alpha_{\uparrow} n/\mathit{gaptol},\end{aligned}$$

and the claim follows. $\square$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Now a straightforward argument based on Lemma 4.2 and using Lemma 4.3 inductively allows us to link the computed vectors to invariant subspaces higher up in the tree.

LEMMA 4.4 (computed vectors and the local invariant subspaces). *Let the conditions for Lemmas* 4.2 *and* 4.3 *be fulfilled. Then for each node* (M, I) *in the tree with child index set* $J \subseteq I$*, the computed vectors* $\bar{\mathsf{q}}_j$*,* $j \in J$*, will obey*

$$\sin\angle\big(\bar{\mathsf{q}}_j\,,\,\mathcal{Q}_J[\mathsf{M}]\big) \;\le\; \mathcal{R}n\epsilon_\diamond + C_{\text{vecs}}(\mathrm{d}(j)-\mathrm{d}(\mathsf{M}))(\alpha_\downarrow+\alpha_\uparrow)n/gaptol,$$

with $\mathcal{R}$ *as in* (4.2) *and* $\mathrm{d}(j)$ *and* $\mathrm{d}(\mathsf{M})$ *as in section* 3.3.

Proof. If the eigenvalue j is a singleton, $J = \{j\}$, then $\mathrm{d}(j) = \mathrm{d}(\mathsf{M})$, and the claim is just Lemma 4.2.

For the case $|J| > 1$, let (H, J) be the corresponding child node and τ be the shift leading to it. Fix any index $j \in J$ and let N be the unique child index set of J that contains j. Note that N may or may not be a singleton. In any case,

$$\begin{aligned}
&\sin\angle\big(\bar{\mathsf{q}}_j\,,\,\mathcal{Q}_J[\mathsf{M}]\big)\\
&\quad\le \sin\angle\big(\bar{\mathsf{q}}_j\,,\,\mathcal{Q}_J[\mathsf{H}]\big) + \sin\angle\big(\mathcal{Q}_J[\mathsf{H}]\,,\,\mathcal{Q}_J[\mathsf{M}]\big) && \text{using Lemma 2.2,}\\
&\quad\le \sin\angle\big(\bar{\mathsf{q}}_j\,,\,\mathcal{Q}_N[\mathsf{H}]\big) + \sin\angle\big(\mathcal{Q}_J[\mathsf{H}]\,,\,\mathcal{Q}_J[\mathsf{M}]\big) && \text{by Lemma 2.1, as } N \subseteq J.
\end{aligned}$$

The second term on the right-hand side is handled by Lemma 4.3. For the first term, we employ the induction hypothesis, noting that $j \in N$, N is a child index set of J, and $\mathrm{d}(\mathsf{H}) = \mathrm{d}(\mathsf{M}) + 1$; together they complete the argument. ∎

Now we can quantify the notion raised at the beginning of section 3.2, in (3.3) and (3.4), namely that accuracy yields orthogonality.

THEOREM 4.5 (orthogonality of MR^3). *Let the representation tree traversed by Algorithm MR*^{3} *satisfy the conditions for Lemma* 4.4*, and let* $\mathrm{d}_{\max} := \max\{\mathrm{d}(i) \,|\, i \in I_0\}$ *be the maximum depth of a node in the tree. Then any two computed vectors* $\bar{\mathsf{q}}_i$ *and* $\bar{\mathsf{q}}_j$*,* $i \ne j$*, will obey*

$$\tfrac{1}{2}|\bar{\mathsf{q}}_i^*\bar{\mathsf{q}}_j| \;\le\; \mathcal{R}n\epsilon_\diamond + C_{\text{vecs}}\mathrm{d}_{\max}(\alpha_\downarrow+\alpha_\uparrow)n/gaptol,$$

with $\mathcal{R}$ *as in* (4.2).

Proof. Let (M, N) be the unique deepest ancestor of both i and j, meaning that N has child index sets $I \ne J$ with $i \in I$ and $j \in J$. From Lemma 4.4 we know that $\bar{\mathsf{q}}_i$ and $\bar{\mathsf{q}}_j$ can be written as

$$\begin{aligned}
\bar{\mathsf{q}}_i &= \mathsf{x}+\mathsf{r}, \quad \mathsf{x} \in \mathcal{Q}_I[\mathsf{M}], \quad \mathsf{x} \perp \mathsf{r}, \quad \|\mathsf{r}\| = \sin\angle\big(\bar{\mathsf{q}}_i\,,\,\mathcal{Q}_I[\mathsf{M}]\big),\\
\bar{\mathsf{q}}_j &= \mathsf{y}+\mathsf{s}, \quad \mathsf{y} \in \mathcal{Q}_J[\mathsf{M}], \quad \mathsf{y} \perp \mathsf{s}, \quad \|\mathsf{s}\| = \sin\angle\big(\bar{\mathsf{q}}_j\,,\,\mathcal{Q}_J[\mathsf{M}]\big),
\end{aligned}$$

and we have bounds on $\|\mathsf{r}\|$ and $\|\mathsf{s}\|$. The situation is depicted in Figure 4.2. Hence,

$$\bar{\mathsf{q}}_i^*\bar{\mathsf{q}}_j \;=\; \underbrace{\mathsf{x}^*\mathsf{y}}_{=0} + \mathsf{x}^*\mathsf{s} + \mathsf{r}^*\mathsf{y} + \mathsf{r}^*\mathsf{s} \;=\; \mathsf{r}^*\bar{\mathsf{q}}_j + \mathsf{x}^*\mathsf{s} \quad (=\; \bar{\mathsf{q}}_i^*\mathsf{s} + \mathsf{r}^*\mathsf{y}),$$

and as we assume $\|\bar{\mathsf{q}}_i\| = \|\bar{\mathsf{q}}_j\| = 1$,

$$\big|\bar{\mathsf{q}}_i^*\bar{\mathsf{q}}_j\big| \;\le\; \big|\mathsf{r}^*\bar{\mathsf{q}}_j\big| + |\mathsf{x}^*\mathsf{s}| \;\le\; \|\mathsf{r}\| + \|\mathsf{s}\|,$$

where we had to invoke the Cauchy–Schwarz inequality for the last step. ∎

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

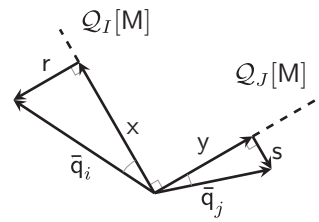

FIG. 4.2. *Situation for the proof of Theorem* 4.5.

This completes what we set out to do. It should be noted that the error bounds are all of a worst-case nature. The accuracy delivered by MR^3 is in general far better than these would suggest, as we will see in section 6. Nevertheless they are also sharp in the following sense: Running MR^3 with a gap tolerance of, say, *gaptol* $= 0.001$, one must be prepared for orthogonality levels of about $\mathcal{O}(1000n\epsilon_\diamond)$, because they can occur even if all of the requirements are fulfilled with very benign parameters.

5. Towards an improved implementation of MR^3. MR^3 is an algorithm that can be explained rather compactly if the details are left out; see section 3. But when getting to the subcomponents—like the iteration for computing vectors, or finding a good shift—working just "right" is a tricky business.

In this section we will briefly describe how some of these aspects are handled in our MR^3 implementation `XMR` and in particular point out where it differs significantly from `DSTEMR` in LAPACK 3.2.2, which is—at the time of this writing—still the most recent implementation. The `XMR` code can be obtained from http://www-ai.math.uni-wuppertal.de/SciComp/XMR.

The reader is warned that some experience with MR^3 will be necessary to follow the discussion of the algorithmic variants in this section. Novices to MR^3 and readers not interested in these issues may prefer to go directly to the numerical results in section 6.

5.1. Issues pertaining to any MR^3 implementation. First we would like to raise some more general points that pertain to any implementation of the algorithm. More specific information will be given in the remaining subsections.

Computation of eigenvalues (line 5). Because only a subset of eigenvalues is to be computed, some variant of the bisection method is advisable to get intervals $[\lambda_i^{\text{loc}}]$ for each eigenvalue. The accuracy need only be good enough to enable reliable classification; e.g., for *gaptol* $= 0.001$ getting about the first three decimal digits would suffice. For clusters the bounds can then be inflated a little and shifted by τ to serve as useful starting points for the eigenvalue refinement at child nodes.

Computation of eigenvectors (line 8). At this point an initial approximation to the eigenvalue will be available from the classification step. In principle one could again use bisection to refine the eigenvalue to full accuracy before computing the vector, but for faster convergence a Rayleigh quotient iteration (RQI) is usually deployed in practice. The eigenpair is improved until the residual norm is driven below some threshold, for instance until requirement GETVEC from section 4 is fulfilled.

Shifting (line 12). This is one of the most critical and also most subtle steps in the whole algorithm. On the one hand, we need to shift as close as possible or even inside the cluster to break up the relative gaps. For this it is usually reasonable to further refine at least the intervals for the cluster boundaries following the classifica-

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

tion phase. On the other hand, the representation of the shifted matrix has to satisfy the requirements that have been compiled in section 4.2, and one may have to back off from the cluster to fulfill these.

The set $\mathcal{S}$. The management of $\mathcal{S}$ determines the order in which the representation tree is traversed. This is irrelevant for the algorithm itself, but there are practical considerations. A *breadth-first* strategy can save workspace memory by storing the $\leq 2n-1$ data elements of the node representations in the eigenvector matrix, because each node covers at least two eigenvalues. This is how MR^3 was designed originally [11]. Alternatively a *depth-first* strategy can be employed, as proposed, for example, in [17]. The advantage is that it provides the option of backtracking, should insurmountable problems within a subtree arise—for instance that no suitable shift candidate can be found. Also, the copying of representation data to and from the eigenvector matrix is avoided. The only disadvantage of a *depth-first* strategy is that it requires capping the maximum depth of the tree (to, say, 10) so that all representations along the longest path can be stored together in a reserved area of workspace.

5.2. How to achieve the requirements. We know that any implementation of MR^3 has to heed the five requirements in section 4.2. Three of those are actually of no concern:

- RELGAPS comes mostly for free, provided that the representations are robust in even the most rudimentary terms, and the implied conditions for the outer gaps are easily checked.
- SHIFTREL is fulfilled if the routine to compute the child representation in line 12 is componentwise mixed relatively stable. This goal can be achieved, e.g., with twisted factorizations or block factorizations [9, 38, 39].
- GETVEC is not always easy to achieve, but given a faithful residual norm for the vector, fulfillment of the requirement can be monitored during runtime. If the residual is not good enough, the eigenvalue approximation can be refined, e.g., using the Rayleigh quotient correction; cf. the remarks above.

The remaining two, RRR and ELG, have to be attained in line 12 of Algorithm MR^3, where the shifts are selected. The problem with them is that *we cannot always know a priori, that is, at the time of computing the child representation, whether the requirements are met.* Sometimes this is possible to know for sure only a posteriori, *after* the vectors have been computed.

The fact that the vectors are indeed needed for evaluation is evident from how ELG is stated. In order to fulfill RRR (and GETVEC), we need to be able to control the effect that any erp

$$\mathsf{T} = \mathsf{N}\mathsf{G}\mathsf{N}^* \rightsquigarrow \widetilde{\mathsf{N}}\widetilde{\mathsf{G}}\widetilde{\mathsf{N}}^* = \widetilde{\mathsf{T}} = \mathrm{erp}(\mathsf{T}, \alpha)$$

(stated here for twisted factorizations) can have on the local subset of eigenpairs. Following along the lines of [26, 27, 30, 36] it can be shown [37, section 2.3] that to first order the change of an eigenvalue λ_j is bounded as

$$|\delta\lambda_j| \;\leq\; \alpha\, \|\,|\mathsf{G}|^{1/2}\mathsf{N}^*\mathsf{q}_j\|^2 \;=\; \alpha\, \mathsf{q}_j^*\mathsf{N}|\mathsf{G}|\mathsf{N}^*\mathsf{q}_j,$$

where q_j is a corresponding normalized eigenvector, and $|\mathsf{G}| = \Omega\mathsf{G}$ with a diagonal signature matrix $\Omega = \mathrm{diag}(\pm 1)$. Noting that $\mathsf{q}_j^*\mathsf{N}\mathsf{G}\mathsf{N}^*\mathsf{q}_j = \mathsf{q}_j^*\mathsf{T}\mathsf{q}_j = \lambda_j$, it therefore makes sense to define the *relative condition number* (relcond) of an eigenvalue λ_j via

$$\kappa_{\mathrm{rel}}(\lambda_j) \;:=\; \frac{\mathsf{q}_j^*\mathsf{N}|\mathsf{G}|\mathsf{N}^*\mathsf{q}_j}{|\mathsf{q}_j^*\mathsf{N}\mathsf{G}\mathsf{N}^*\mathsf{q}_j|}. \tag{5.1}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

This is how relative condition numbers were originally defined by Dhillon for LDL^* decompositions [7]. The relative condition again exposes the fact that the eigenvectors may be necessary in order to check RRR. We now will discuss two criteria that can be tested a priori; see (5.3) and (5.4) below.

The bounds that can be inferred without eigenvector information are more often than not too crude to be useful. However, it turns out that to obtain practicable a priori bounds, it usually suffices to know where *all* vectors in the local invariant subspace of interest have small entries. This information is captured in the *envelope* of the subspace, which is formally defined as the vector $\mathrm{env}_{\mathcal{S}} \in \mathbb{R}^n$ with entries

$$\mathrm{env}_{\mathcal{S}}(i) \;:=\; \max\big\{|\mathsf{x}(i)| \,:\, \mathsf{x} \in \mathcal{S}, \|\mathsf{x}\| = 1\big\}.$$

The amount of accurate information that can be determined cheaply about an envelope can sometimes be astonishing [25, 35].

We see that the components of the envelope are upper bounds on entries of any unit vector from the subspace. If some of them are small, we say there is *eigenvector localization*. Localization can reasonably be expected only for clusters $I = \{c, \ldots, d\}$ that are small, say $|d - c| < n/3$, as well as *tight* in the sense that

$$|\lambda_c - \lambda_d| \;\le\; \mathit{tfac}\,\mathrm{gap}_{\mathsf{T}}(I), \quad \mathit{tfac} \ll 1, \tag{5.2}$$

with, for instance, $\mathit{tfac} = 2^{-8}$. That is, the width of the cluster is substantially smaller than the distance to the nearest eigenvalue.

Suppose we have an approximation s for $\mathrm{env}_{\mathcal{S}}$. It should be faithful in the sense that $\mathrm{env}_{\mathcal{S}}(i) \le \mathsf{s}(i) \le 1$ for all entries, but it might have $\mathsf{s}(i) \equiv 1$ if no useful information can be deduced.

For a twisted factorization $\mathsf{M} = \mathsf{N}\mathsf{G}\mathsf{N}^*$ with twist index k as representation, write

$$(\mathsf{N} - \mathsf{I})\mathsf{G}(\mathsf{N} - \mathsf{I})^* = \mathsf{U} + \mathsf{V},$$

where U and V are diagonal matrices with sole nonzero entries $\mathsf{U}(i,i) = \ell_{i-1}^2 d_{i-1}$ for $1 < i \le k$ and $\mathsf{V}(i,i) = u_{i+1}^2 r_{i+1}$ for $k \le i < n$. Then the second condition in ELG can be replaced by

$$\|\mathsf{G}\mathsf{s}\| + \|\mathsf{U}\mathsf{s}\| + \|\mathsf{V}\mathsf{s}\| \;\le\; C_{\mathrm{elg}}\,\mathrm{spdiam}[\mathsf{M}_0] \tag{5.3}$$

with a slightly modified C_{elg}; cf. [9, Lemma 4] for a proof.

For requirement RRR, the formula (5.1) for relconds is not well suited to be used with an envelope. A better one can be found in [30, p. 126]; we adapt it now to twisted factorizations. We note that

$$\mathsf{q}_j^*\mathsf{N}|\mathsf{G}|\mathsf{N}^*\mathsf{q}_j \;=\; \mathsf{q}_j^*\mathsf{N}\Omega\mathsf{N}^{-1}\mathsf{N}\mathsf{G}\mathsf{N}^*\mathsf{q}_j \;=\; \mathsf{q}_j^*\mathsf{N}\Omega\mathsf{N}^{-1}\mathsf{q}_j\lambda_j,$$

which eliminates the explicit dependence on G. This implies

$$\kappa_{\mathrm{rel}}(\lambda_j) \;=\; |\mathsf{q}_j^*\mathsf{N}\Omega\mathsf{N}^{-1}\mathsf{q}_j| \;\le\; \|\Upsilon\mathsf{N}^*\mathsf{q}_j\|\|\Upsilon^{-1}\mathsf{N}^{-1}\mathsf{q}_j\|$$

for any (nonsingular) diagonal matrix Υ. Hence, requirement RRR can be checked a priori via the condition

$$\|\Upsilon\mathsf{N}^*\mathsf{s}\|\|\Upsilon^{-1}\mathsf{N}^{-1}\mathsf{s}\| \;\le\; \mathit{maxrc}, \tag{5.4}$$

where Υ can be chosen freely to minimize the resulting bound, and *maxrc* is a parameter of moderate size, say 10. Note that, due to N being composed of two bidiagonal parts, this bound can be evaluated with $\mathcal{O}(n)$ multiplications and additions.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

5.2.1. Outline of DSTEMR. For comparison purposes, let us first describe in a few words how the routine DSTEMR in LAPACK 3.2.2 was set up; for more information see [11] or refer directly to the source code available from http://www.netlib.org/lapack.

The basic design is faithfully depicted by Algorithm MR^3 in Figure 3.1. Only plain LDL^* factorizations are used as representations at nodes.

Concerning selection of shifts in line 12, DSTEMR refines the eigenvalues at the borders of clusters to full precision and then tries to shift close to one of them, but still outside, backing off if the first tries are not successful. To evaluate a shift, DSTEMR checks only for element growth, that is, condition (5.3) above, with $C_{\text{elg}} = 8$. If no candidate satisfies the condition, DSTEMR takes the one with minimal element growth. Envelope information is obtained in a very restricted fashion: For a shift on the left, DSTEMR uses a rough approximation to the "rightmost" eigenvector of the cluster as envelope, and vice versa. This approach is very efficient but runs the danger of using wrong envelope information. To alleviate this concern, it is employed only if the plain element growth (unweighted) still lies below a safeguard threshold.

Once a shift has been selected and the corresponding representation is computed, DSTEMR will initialize the child's eigenvalue bounds. Let $[\lambda_i]$ be the bounds at the parent; then

$$[\lambda_i^+] \;:=\; [\lambda_i] - \tau$$

become the bounds for the child. Thus, before classification starts in line 5 for the child, the child bounds are inflated until they have consistent Sturm counts.

5.2.2. Outline of our implementation XMR. The basic principle guiding the design of our implementation XMR was reliability. One cannot completely avoid situations where no shift candidate is acceptable a priori, in particular if no useful envelope is available. Furthermore we made the experience that the sole testing of element growth, like DSTEMR does, can lead one astray: for some (admittedly rare, synthetic) test cases we encountered two candidates with comparable, acceptable element growth, but manual experimentation revealed only one of them to be relatively robust.

Hence we want to accumulate as much information about the candidates as possible, to make a better-informed choice. This is why we included the a priori checking of condition (5.4) for relative robustness.

For the same reason we restructured Algorithm MR^3 a little by incorporating some bisection steps directly into line 12. Once a shift is deemed worthy of selection because it seems best—either passing the a priori tests or failing them less badly than the other candidates—the shifted representation is computed and initial bounds for its local eigenvalues are set up. Let $[\lambda_i]$, $i \in I$, be the local approximations (intervals) at the parent. Then we initialize the bounds for the child as

$$[\lambda_i^+] \;:=\; \big([\lambda_i](1 \pm \alpha) - \tau\big)(1 \pm \alpha), \tag{5.5}$$

where α is a parameter (currently $\alpha = 100n\epsilon_\diamond$). In our experience, *one of the first signs that something is wrong is that the eigenvalues start to jump out of even these inflated intervals.* Hence the bounds are then "verified" on the spot to be consistent by doing two Sturm counts each. Should they prove to be inconsistent, the candidate is immediately discarded. In this case the Sturm counts could be seen as wasted, but they did prevent us from making a wrong choice. However, if the bounds pass basic consistency, we can save the (costly) inflation of the child bounds that DSTEMR has

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

to do. We are then also in a position to use just two more Sturm counts to "verify" that the outer relative gaps at the child exceed *gaptol*, thus establishing that part of RELGAPS.

The refined criteria from section 5.2 also allow for the integration of another technique into our code, which is to exploit the *sub*structure of a cluster. As a motivating example consider a cluster of 20 eigenvalues with

$$\lambda_1, \ldots, \lambda_{10} \doteq 1 + \mathcal{O}(\epsilon_\diamond), \qquad \lambda_{11}, \ldots, \lambda_{20} \doteq 1 + gaptol/2 + \mathcal{O}(\epsilon_\diamond).$$

We have to treat all 20 as one cluster, although we would like to separate the two subgroups. Considering envelopes, the condition (5.2) indicates that both subgroups may have significant eigenvector localization, and thus the whole cluster may have it, too, although (5.2) considered only for the whole cluster would make us think otherwise.

Our general strategy is to refine eigenvalues in clusters to reveal interior gaps exceeding $\sqrt{\epsilon_\diamond}$. Then we try to find envelopes for each subgroup and combine them to form an envelope for the whole cluster. To construct the envelopes for the subgroups we have implemented the new techniques from [35]. This approach gives us usable and faithful envelope information for many cases where `DSTEMR` cannot use any.

Once cluster substructure has been determined, it is tempting to exploit it further by also trying to shift inside the cluster. It was feared before that doing so runs more risk of coming too close to Ritz values, thereby causing element growth. And indeed, just blindly taking the cluster midpoint is most often a fatal decision (we tried). However, in our experience inside shifts can work very well, *as long as* one makes sure to keep the representations nearly singular, that is, place the shift within an interior gap, close to a boundary. In the example above we would try to shift slightly above λ_{10} or slightly below λ_{11}, additionally to trying shifts on the outside.

Although this use of cluster substructure indeed works, in the current version of our code we have deactivated the option for performance reasons, since all our test cases could be solved to satisfaction without it.

Further significant differences to `DSTEMR` are the following:

- We use *twisted block factorizations* as representations at the nodes, which provide for greatly reduced element growth and relative robustness. Twisted block factorizations generalize the "standard" twisted factorizations (cf. section 3.2) by combining the upper part of a block LDL^* factorization and the bottom part of a block URU^* factorization, both of which may contain blocks of size up to 2×2 [38]. However, the twist pivot γ_k is always 1×1. The twist index is chosen on a case-by-case basis to minimize the bounds for both conditions (5.3) and (5.4) simultaneously. Eigenvectors are still computed using plain (unblocked) twisted factorizations; that is, in (4.1) the representation M has blocks, but G_k is diagonal.
- The tolerances for RQI and bisection were initially taken from `DSTEMR` and then optimized. In particular for singletons we switch from bisection to RQI far earlier to benefit from the faster convergence. Furthermore we noted that the acceptance thresholds in RQI are rather strict in `DSTEMR`, causing the iteration to continue (or, worse, fallback bisection to be invoked) needlessly sometimes.
- We use a different bisection design for `XMR`. The bisection process keeps a list of nonoverlapping intervals that contain one or more eigenvalues each. For instance, the child bounds in (5.5) are set up using the bounds in that list, and not for each eigenvalue separately.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

By contrast, DSTEMR uses a simpler scheme, with one interval per eigenvalue. This can be rather wasteful if the eigenvalues are close together, since overlapping intervals are refined independently.

Finally we have discovered rare cases where as yet unregarded *underflow* in the qd-transformations can lead to loss of relative accuracy. Going into detail on this issue would be beyond the scope of this paper, so we can give only a short description here; more details can be found in [39, section 8.2].

Up to now the only safeguard put in place within the qd-algorithms concerned protection against underflow in the pivots. This was done by deflecting small pivots from zero using a suitable threshold *pivmin*, or alternatively by exploiting IEEE 754 arithmetic; cf. [9, 39]. The new cause for loss of relative accuracy does not concern the pivots, but rather is due to underflow during a sequence of multiplications and divisions. By adapting the *pivmin*-scheme we have made our code XMR completely underflow-safe and independent of IEEE 754 compliant handling of NaNs or exceptions.

We mentioned that these cases are rare, and furthermore they are hard to track down. However, we have at least one test case where we are sure that exactly this underflow issue causes DSTEMR to return a zero vector, without signaling an error code. The loss of relative accuracy leads to a twist that is completely wrong and in turn to a vector of huge norm that overflows.

6. Numerical results. To compare our custom implementation XMR (with twisted block factorizations) with the solvers DSTEDC and DSTEMR from LAPACK, we have prepared three disjunct testsets of symmetric tridiagonal matrices.

Since the main objective of our research was to increase the robustness of MR^3, that is, to reduce the probability that MR^3 fails either by not being able to compute some of the eigenpairs or by returning eigenpairs with unacceptable residuals or orthogonality levels, two testsets are used to assess the quality of the results.

The testset Pract contains 56 matrices with dimensions up to 6245. They stem from various applications, and we obtained them from Osni Marques and Christof Vömel. More information about the specific matrices can be found in [6], where the same set was used to evaluate the symmetric eigensolvers in LAPACK. The nature of the set Pract is that we expect it to represent the kinds of problems that could be expected in actual applications. This does not mean that these problems are easy to solve, far from it. Nevertheless we will see that the really "hard" cases are in Synth.

The testset Synth was designed to contain matrices with very tight clusters in various highly demanding settings, in order to provide a real stress-test for MR^3 with respect to robustness. Note that, except for obvious errors, faulty behavior may occur for one matrix size, whereas a slightly smaller or larger problem runs just fine (even for the same type of eigenvalue distribution). Therefore Synth is quite comprehensive, with 116874 matrices that were generated according to the following scheme:

(1) We have implemented a base collection of synthetic matrix types, which can in principle be generated for any dimension. Included are the well-known standard types like Wilkinson and Clement matrices, as well as matrices generated from given eigenvalue distributions (extending those built into LAPACK's test matrix generator DLATMS). In fact, all artificial types listed in [23] are present.
(2) For all types from (1), all matrices for dimensions $2, \ldots, 200$ were generated. If the type depends on randomness, four versions based on different random number seeds were produced.
(3) Each matrix resulting from step (2) was taken in three variants: as is, glued

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 6.1

Orthogonality levels $|\mathsf{Q}^*\mathsf{Q} - \mathsf{I}|/n\epsilon_\diamond$ and residual norms $\|\mathsf{T}\mathsf{q} - \mathsf{q}\lambda\|/\|\mathsf{T}\|n\epsilon_\diamond$ between our implementation with nonblocked or blocked twisted factorizations (`XMR-nb` and `XMR`, respectively) and the LAPACK *solvers `DSTEMR` and `DSTEDC` over the* Pract *and* Synth *testsets. An empty entry stands for zero. FAIL means the routine returned an error code, whereas WRONG stands for an effectively useless result without a warning. The results were capped at the value WRONG, meaning the* 0.636% *from* Synth *where `DSTEMR` exceeded* 10^9 *for orthogonality were not included for computing the avg, med, and max statistics.*

Pract (56 cases)				**Synth** (116874 cases)			
DSTEDC	DSTEMR	XMR		DSTEDC	DSTEMR	XMR-nb	XMR
Orthogonality level $|\mathsf{Q}^*\mathsf{Q} - \mathsf{I}|/n\epsilon_\diamond$							
0.09	13.74	3.48	AVG	1617	49030	234.19	1.44
0.04	3.84	1.71	MED	0.06	0.80	0.49	0.48
0.50	313.36	33.95	MAX	$16.7 \cdot 10^6$	$99.5 \cdot 10^6$	$9.68 \cdot 10^6$	9473
100 %	75 %	91 %	0 . . . 10	99.916 %	90.193 %	98.724 %	98.894 %
	23 %	9 %	10 . . . 100	0.013 %	6.073 %	1.119 %	1.070 %
	2 %		100 . . . 1000	0.009 %	1.397 %	0.077 %	0.030 %
			> 1000	0.050 %	1.639 %	0.080 %	0.006 %
			WRONG ($\geq 10^9$)		0.636 %		
			FAIL	0.012 %	0.062 %		
Residual norms $\|\mathsf{T}\mathsf{q} - \mathsf{q}\lambda\|/\|\mathsf{T}\|n\epsilon_\diamond$							
0.07	0.44	0.41	AVG	0.07	1.71	0.63	0.63
0.01	0.03	0.07	MED	0.05	0.07	0.13	0.13
0.60	3.73	4.67	MAX	1.34	82526	13.48	13.48
100 %	88 %	84 %	0 . . . 1	99.979 %	97.603 %	80.807 %	80.807 %
	12 %	16 %	1 . . . 10	0.009 %	2.308 %	19.192 %	19.192 %
			10 . . . 100		0.007 %	0.001 %	0.001 %
			> 100		0.014 %		
			WRONG ($\geq 10^5$)		0.006 %		
			FAIL	0.012 %	0.062 %		

once to itself with a small glue ($\sim n\epsilon_\diamond\|\mathsf{T}\|$), and glued twice to itself with medium glue ($\sim n\sqrt{\epsilon_\diamond}\|\mathsf{T}\|$). For more details on the intricacies of glued matrices we refer the reader to [10, 32].

Due to its comprehensive nature, Synth also contains many cases that are borderline trivial.

Table 6.1 summarizes the orthogonality levels and residual norms of the three methods `DSTEDC`, `DSTEMR`, and `XMR` on the testsets Pract and Synth. As can be seen, all methods handle the problems in Pract reasonably well. It becomes evident that the hard cases are indeed in Synth, and here is also where our efforts to improve reliability for `XMR` show effect. There are several cases in there for which `DSTEMR` does not produce satisfactory results, and even `DSTEDC` is sometimes brought to its limits. We can state that the MR^3-based methods cannot provide the accuracy of DC. Concerning robustness, `XMR` clearly outperforms `DSTEMR`. None of the test matrices in Synth was able to break `XMR`, and the worst-case errors are reduced substantially. The table also indicates that this holds even if the twisted block factorizations are disabled and standard twisted factorizations are used instead (column `XMR-nb`), although the blocked factorizations provide a noticeable further improvement. For less demanding matrices, the eigenpairs produced with `XMR` and `XMR-nb` feature somewhat higher residuals, but better orthogonality, than those from `DSTEMR`.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

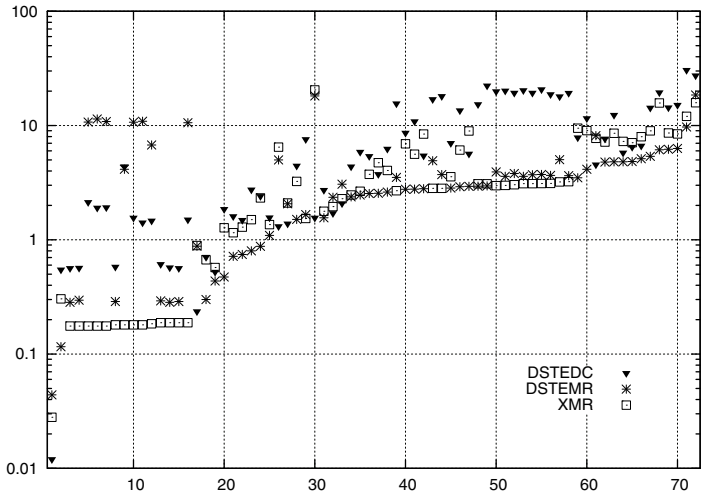

FIG. 6.1. *Absolute timings in seconds of the two* LAPACK *solvers* `DSTEDC` *and* `DSTEMR` *and our implementation* `XMR` *over the testset* Large. *The test cases are sorted by "difficulty," i.e., with respect to the time taken by the fastest method.*

To obtain meaningful timings a third set Large is composed of two groups of large matrices, all of them generated with `DLATMS`. First, for each matrix in Pract with dimension exceeding 1000, Large contains a matrix of the same size and eigenvalue distribution. And second, for each synthetic type in (1) above we generated a matrix of dimension about 5000.

The timings shown in Figure 6.1 for the testset Large were obtained on a 3 GHz Core 2 Quad Q9650 machine with 4 GBytes of main memory, using the reference implementation of LAPACK 3.2.2 compiled with version 4.3.2 of the GNU Fortran compiler and linked with the Intel MKL for BLAS routines. They indicate that the increased robustness is not bought by a severe decrease in speed. In fact, the average timings are 3.99 seconds for `XMR` and 4.09 seconds for `DSTEMR`, and in 32 out of the 72 cases `XMR` was faster. `DSTEDC` takes 8.07 seconds on average, and the data suggest that the MR^3-based routines tend to be faster for larger matrices. This trend is confirmed by a matrix of dimension 27,069 arising in self-consistent field (SCF) computations that takes 1,561 (231, 245) seconds with `DSTEDC` (`DSTEMR` and `XMR`, respectively) on a machine with two six-core X5680 Xeon processors and 96 GBytes of main memory, again using a self-compiled LAPACK reference implementation together with the Sun Performance Library for BLAS. (On this machine, setting `PARALLEL=12` to make use of the parallelized matrix-matrix products within the optimized BLAS reduces the `DSTEDC` time to 220 seconds, whereas the `XMR` code allows computing the eigenpairs independently, but consistently, in 12 subsets, with a maximum of 28 seconds for one subset.)

A final remark concerning the possible danger of testset bias: We did not use the above testsets for testing and tuning our software; at least the synthetic matrices we used for these purposes were fewer and generated with a different random seed.

Acknowledgments. The authors want to thank Beresford Parlett, who strongly influenced the work leading to this paper, Christof Vömel and Osni Marques for

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

making available to us the matrices from which the testset Pract was constructed, and the anonymous referees for their valuable comments, which helped improve the presentation. This work was carried out while P. Willems was with the Faculty of Mathematics and Natural Sciences at the University of Wuppertal.

REFERENCES

[1] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide*, 3rd ed., SIAM, Philadelphia, 1999.

[2] P. Bientinesi, I. S. Dhillon, and R. A. van de Geijn, *A parallel eigensolver for dense symmetric matrices based on multiple relatively robust representations*, SIAM J. Sci. Comput., 27 (2005), pp. 43–66.

[3] J. J. M. Cuppen, *A divide and conquer method for the symmetric tridiagonal eigenproblem*, Numer. Math., 36 (1981), pp. 177–195.

[4] J. W. Demmel, *Applied Numerical Linear Algebra*, SIAM, Philadelphia, 1997.

[5] J. W. Demmel and W. Kahan, *Accurate singular values of bidiagonal matrices*, SIAM J. Sci. Comput., 11 (1990), pp. 873–912.

[6] J. W. Demmel, O. A. Marques, B. N. Parlett, and C. Vömel, *Performance and accuracy of LAPACK's symmetric tridiagonal eigensolvers*, SIAM J. Sci. Comput., 30 (2008), pp. 1508–1526.

[7] I. S. Dhillon, *A New $O(n^2)$ Algorithm for the Symmetric Tridiagonal Eigenvalue/Eigenvector Problem*, Ph.D. thesis, University of California, Berkeley, Berkeley, CA, 1997.

[8] I. S. Dhillon and B. N. Parlett, *Multiple representations to compute orthogonal eigenvectors of symmetric tridiagonal matrices*, Linear Algebra Appl., 387 (2004), pp. 1–28.

[9] I. S. Dhillon and B. N. Parlett, *Orthogonal eigenvectors and relative gaps*, SIAM J. Matrix Anal. Appl., 25 (2004), pp. 858–899.

[10] I. S. Dhillon, B. N. Parlett, and C. Vömel, *Glued matrices and the MRRR algorithm*, SIAM J. Sci. Comput., 27 (2005), pp. 496–510.

[11] I. S. Dhillon, B. N. Parlett, and C. Vömel, *The design and implementation of the MRRR algorithm*, ACM Trans. Math. Software, 32 (2006), pp. 533–560.

[12] K. V. Fernando, *On computing an eigenvector of a tridiagonal matrix. Part* I: *Basic results*, SIAM J. Matrix Anal. Appl., 18 (1997), pp. 1013–1034.

[13] K. V. Fernando and B. N. Parlett, *Accurate singular values and differential qd algorithms*, Numer. Math., 67 (1994), pp. 191–229.

[14] J. Francis, *The QR transformation: A unitary analogue to the LR transformation.* I., Comput. J., 4 (1961/1962), pp. 265–271.

[15] J. Francis, *The QR transformation.* II., Comput. J., 4 (1961/1962), pp. 332–345.

[16] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 3rd ed., Johns Hopkins University Press, Baltimore, MD, 1996.

[17] B. Grosser, *Ein paralleler und hochgenauer $O(n^2)$ Algorithmus für die bidiagonale Singulärwertzerlegung*, Ph.D. thesis, Bergische Universität Gesamthochschule Wuppertal, Fachbereich Mathematik, Wuppertal, Germany, 2001.

[18] M. Gu and S. C. Eisenstat, *A divide-and-conquer algorithm for the symmetric tridiagonal eigenproblem*, SIAM J. Matrix Anal. Appl., 16 (1995), pp. 172–191.

[19] I. C. F. Ipsen, *Computing an eigenvector with inverse iteration*, SIAM Rev., 39 (1997), pp. 254–291.

[20] W. Kahan, *Accurate Eigenvalues of a Symmetric Tri-diagonal Matrix*, Technical Report CS41, Computer Science Department, Stanford University, Stanford, CA, 1966.

[21] B. Lang, *Using level* 3 *BLAS in rotation-based algorithms*, SIAM J. Sci. Comput., 19 (1998), pp. 626–634.

[22] O. A. Marques, B. N. Parlett, and C. Vömel, *Computations of eigenpair subsets with the MRRR algorithm*, Numer. Linear Algebra Appl., 13 (2006), pp. 643–653.

[23] O. A. Marques, C. Vömel, J. W. Demmel, and B. N. Parlett, *Algorithm* 880: *A testing infrastructure for symmetric tridiagonal eigensolvers*, ACM Trans. Math. Software, 35 (2008), pp. 8:1–8:13.

[24] B. N. Parlett, *The Symmetric Eigenvalue Problem*, Prentice–Hall, Englewood Cliffs, NJ, 1980.

[25] B. N. Parlett, *Invariant subspaces for tightly clustered eigenvalues of tridiagonals*, BIT, 36 (1996), pp. 542–562.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[26] B. N. Parlett, *Spectral sensitivity of products of bidiagonals*, Linear Algebra Appl., 275/276 (1998), pp. 417–431.

[27] B. N. Parlett, *Perturbation of eigenpairs of factored symmetric tridiagonal matrices*, Found. Comput. Math., 3 (2003), pp. 207–223.

[28] B. N. Parlett, *A bidiagonal matrix determines its hyperbolic SVD to varied relative accuracy*, SIAM J. Matrix Anal. Appl., 26 (2005), pp. 1022–1057.

[29] B. N. Parlett and I. S. Dhillon, *Fernando's solution to Wilkinson's problem: An application of double factorization*, Linear Algebra Appl., 267 (1997), pp. 247–279.

[30] B. N. Parlett and I. S. Dhillon, *Relatively robust representations of symmetric tridiagonals*, Linear Algebra Appl., 309 (2000), pp. 121–151.

[31] B. N. Parlett and O. A. Marques, *An implementation of the dqds algorithm (positive case)*, Linear Algebra Appl., 309 (2000), pp. 217–259.

[32] B. N. Parlett and C. Vömel, *The spectrum of a glued matrix*, SIAM J. Matrix Anal. Appl., 31 (2009), pp. 114–132.

[33] F. G. Van Zee, R. A. van de Geijn, and G. Quintana-Ortí, *Restructuring the QR Algorithm for High-Performance Application of Givens Rotations*, FLAME Working Note 60, Technical Report TR-11-36, University of Texas at Austin, Austin, TX, 2011.

[34] C. Vömel, *ScaLAPACK's MRRR algorithm*, ACM Trans. Math. Software, 37 (2009), 1.

[35] C. Vömel and B. N. Parlett, *Detecting localization in an invariant subspace*, SIAM J. Sci. Comput. 33 (2011), pp. 3447–3467.

[36] J. H. Wilkinson, *The Algebraic Eigenvalue Problem*, Oxford University Press, Oxford, UK, 1965.

[37] P. R. Willems, *On MR^3-type Algorithms for the Tridiagonal Symmetric Eigenproblem and the Bidiagonal SVD*, Ph.D. thesis, Bergische Universität Wuppertal, Fachbereich Mathematik und Naturwissenschaften, Wuppertal, Germany, 2010.

[38] P. R. Willems and B. Lang, *Block factorizations and qd-type transformations for the MR^3 algorithm*, Electron. Trans. Numer. Anal., 38 (2011), pp. 363–400.

[39] P. R. Willems and B. Lang, *Twisted factorizations and qd-type transformations for the MR^3 algorithm—new representations and analysis*, SIAM J. Matrix Anal. Appl., 33 (2012), pp. 523–553.

Downloaded 12/17/25 to 137.110.35.98 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.